UNIVERSIDAD AUTÓNOMA

GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS

DE LA COMPUTACIÓN Y TELECOMUNICACIONES



Sistema de Gestión de Asistencia Mecánica Vehicular

INTEGRANTES:

- **LOZADA ARIAS VICTOR ADOLFO 221044914**
- **MAMANI PEÑA DANIEL JOAQUIN 222157933**

MATERIA: SISTEMAS DE INFORMACION II -SB

DOCENTE: ING. ROLANDO ANTONIO MARTINEZ CANEDO

NRO GRUPO: 2

SANTA CRUZ - BOLIVIA

Contenido

1 FUNDAMENTACIÓN TEÓRICA 4
1.1 Desarrollo de Aplicaciones Web 4
1.1.1 Métodos de despliegue de aplicaciones 4
1.1.2 Arquitectura basada en capas 4
1.1.3 Arquitectura monolítica vs. microservicios 5
1.2 Desarrollo de Aplicaciones Móviles 5
1.2.1 Enfoques de desarrollo 5
1.2.2 Flutter como framework multiplataforma 5
1.2.3 Integración con servicios backend y funcionalidades del dispositivo 5
1.3 Servicios de Mecánica y Atención de Emergencia 6
1.3.1 Contexto del auxilio mecánico en carretera 6
1.3.2 Modelos de negocio y plataformas digitales 6
1.3.3 Requerimientos del dominio 6
1.4 Backend con Python + FastAPI 6
1.4.1 Características de FastAPI 6
1.4.2 Ventajas para el proyecto: rendimiento asíncrono y ecosistema Python 7
1.4.3 Integración nativa con modelos de IA y servicios de datos 7
1.5 Frontend con Angular 7
1.1.1 Características de Angular 8
1.5.1 Aplicación en el panel web 8
1.5.2 Comunicación con el backend FastAPI 8
1.6 Flutter para Aplicaciones Móviles 8
1.6.1 Arquitectura y lenguaje Dart 8
1.6.2 Widgets y experiencia de usuario 8
1.6.3 Integración de pagos y comunicación con APIs 8
1.7 Inteligencia Artificial: Visión Artificial, Reconocimiento de Audio y Texto 9
1.7.1 Visión artificial para diagnóstico de averías 9
1.7.2 Reconocimiento de audio y procesamiento de lenguaje natural 9
1.7.3 Librerías y servicios (públicos y locales) 9
1.7.4 Modelos supervisados y no supervisados 9
1.7.5 Aplicación en el proyecto: triaje automático del problema 10
2 PUDS 10

2.1 FT:Captura de Requisitos 10
2.1.1 Identificar Actores y Casos de Uso 10
2.1.2 Priorizar Casos de Uso 13
2.1.3 Especificar Casos de Uso 15
2.1.4 Estructurar el modelo de casos de uso 37
2.2 FT:ANALISIS 38
2.2.1 Analisis de arquitectura 38
2.2.2 Analisis de Casos de Uso 44
2.2.3 Analisis de Paquetes 47
2.3 FT: Diseno 48
2.3.1 Diseno de Arquitectura 48
2.3.2 Diseno de Datos 49
2.3.3 Diagrama de secuencia 49
2.4 FT:Implementacion 53
2.4.1 Elección de plataforma de Desarrollo de Software 53
3 Bibliografia 59
4 Anexo 60

1 FUNDAMENTACIÓN TEÓRICA

1.1 Desarrollo de Aplicaciones Web

1.1.1 Métodos de despliegue de aplicaciones

El despliegue de una aplicación web consiste en ponerla a disposición de los usuarios finales en un entorno de producción. Las principales alternativas incluyen:

- **Servidores físicos on-premise:** infraestructura propia, con control total pero altos costes de mantenimiento y escalabilidad limitada.
- **Máquinas virtuales y VPS (Virtual Private Server):** entornos aislados sobre hardware compartido, con mayor flexibilidad.
- **Plataformas como servicio (PaaS):** abstraen la gestión del sistema operativo y middleware (p.ej., Heroku, Google App Engine).
- **Contenedores y orquestación:** Docker + Kubernetes permiten empaquetar la aplicación y escalar horizontalmente.
- **Computación en la nube pública:** servicios como **Azure, Google Cloud y AWS** ofrecen modelos IaaS, PaaS y FaaS (serverless). Brindan alta disponibilidad, balanceo de carga, escalado automático y pago por uso, adaptándose a proyectos con picos de demanda, como una plataforma de emergencias mecánicas que debe estar operativa 24/7.

Para el presente proyecto, se contempla el despliegue en cualquiera de las tres grandes nubes (Azure, Google Cloud, AWS), aprovechando sus servicios gestionados para bases de datos (PostgreSQL), almacenamiento de imágenes y motor de IA.

1.1.2 Arquitectura basada en capas

La arquitectura en capas (layered architecture) organiza el sistema en niveles lógicos con responsabilidades delimitadas, fomentando el bajo acoplamiento y la mantenibilidad. Suele constar de:

- **Capa de presentación (frontend):** interfaz de usuario, en nuestro caso desarrollada con Angular y Flutter.
- **Capa de lógica de negocio (backend):** expone APIs REST, procesa solicitudes y coordina los casos de uso, implementada con **Python + FastAPI**.
- **Capa de acceso a datos:** comunicación con la base de datos PostgreSQL mediante un ORM (SQLAlchemy, Tortoise-ORM) o consultas nativas.
- **Capa de servicios externos:** integración con pasarelas de pago (Stripe, QR), servicios de IA y geolocalización.

Esta separación facilita probar, desplegar y evolucionar cada capa de forma independiente.

1.1.3 Arquitectura monolítica vs. microservicios

- **Arquitectura monolítica:** toda la funcionalidad se empaqueta en un único artefacto desplegable. Es sencilla de desarrollar e inicialmente rápida, pero a medida que crece puede volverse difícil de escalar y mantener.
- **Arquitectura de microservicios:** el sistema se compone de múltiples servicios pequeños, autónomos e independientes, que se comunican mediante protocolos ligeros (HTTP/REST, gRPC, mensajería). Ofrece escalabilidad granular, despliegue continuo y resiliencia, aunque introduce complejidad en la gestión de la red, datos distribuidos y coordinación.

Para una plataforma de auxilio mecánico, una arquitectura de microservicios podría dividir responsabilidades como: servicio de usuarios, servicio de solicitudes de emergencia, servicio de IA (análisis de imágenes/audio/texto), servicio de pagos y servicio de notificaciones. La elección dependerá del volumen de usuarios y la necesidad de escalar componentes concretos. Inicialmente, un monolito modular bien estructurado puede evolucionar hacia microservicios cuando el negocio lo requiera.

1.2 Desarrollo de Aplicaciones Móviles

1.2.1 Enfoques de desarrollo

- **Nativo:** se utilizan los lenguajes y herramientas oficiales de cada plataforma (Kotlin/Java para Android, Swift/Objective-C para iOS). Máximo rendimiento y acceso total a las capacidades del dispositivo, pero mayor coste al mantener dos codebases.
- **Híbrido:** una aplicación web envuelta en un contenedor nativo (Apache Cordova, Ionic). Reutiliza código HTML/CSS/JS, aunque el rendimiento y la experiencia de usuario pueden resentirse.
- **Multiplataforma:** frameworks como Flutter o React Native compilan a código nativo desde una única base de código, equilibrando rendimiento, apariencia nativa y productividad.

1.2.2 Flutter como framework multiplataforma

Flutter, creado por Google, utiliza el lenguaje **Dart** y se basa en un motor de renderizado propio (Skia), lo que garantiza interfaces fluidas a 60/120 fps. Su arquitectura de widgets permite una personalización total y facilita la creación de componentes visuales complejos. Es ideal para aplicaciones que requieren una interfaz atractiva y consistente tanto en Android como en iOS, como la app móvil para solicitar auxilio mecánico.

1.2.3 Integración con servicios backend y funcionalidades del dispositivo

La aplicación móvil se comunicará con el backend **FastAPI** a través de APIs REST o sockets para notificaciones en tiempo real. Además, empleará plugins de Flutter para:

- **Cámara y galería:** captura de imágenes y video de la avería.
- **Micrófono:** grabación de audio que será analizado por IA.

- **Geolocalización:** envío de la ubicación exacta al taller.
- **Pasarelas de pago:** integración de Stripe y generadores de QR para abonar el servicio.

1.3 Servicios de Mecánica y Atención de Emergencia

1.3.1 Contexto del auxilio mecánico en carretera

El auxilio mecánico en carretera abarca desde pinchazos de neumáticos, fallos de batería, problemas de motor, hasta accidentes leves. La rapidez de respuesta es crítica para la seguridad del conductor y la fluidez del tráfico. Tradicionalmente se solicita por llamada telefónica, lo que puede generar demoras en la localización del taller más cercano y en la interpretación del problema.

1.3.2 Modelos de negocio y plataformas digitales

La digitalización ha dado lugar a plataformas que conectan usuarios con prestadores de servicio en tiempo real (similar a Uber, Cabify, pero para grúas y mecánicos). Estas plataformas ofrecen:

- Registro de talleres y mecánicos verificados.
- Solicitud inmediata con geolocalización.
- Seguimiento en vivo del operario.
- Pagos digitales integrados.

1.3.3 Requerimientos del dominio

El proyecto debe abordar:

- **Catálogo de servicios de emergencia:** pinchazos, remolque, recarga de batería, cerrajería, etc.
- **Mecanismo de emparejamiento:** asignación del taller según proximidad, disponibilidad y tipo de avería.
- **Diagnóstico inicial asistido por IA:** usando foto, texto y audio para agilizar la clasificación y preparación del mecánico.
- **Transparencia y trazabilidad:** historial de servicios, facturación y pagos electrónicos (Stripe y códigos QR).

1.4 Backend con Python + FastAPI

1.4.1 Características de FastAPI

FastAPI es un framework web moderno de Python para construir APIs RESTful, basado en estándares como OpenAPI (anteriormente Swagger) y JSON Schema. Sus principales atributos incluyen:

- **Alto rendimiento:** comparable a Node.js y Go, gracias a su núcleo asíncrono construido sobre Starlette y su uso extensivo de asyncio.

- **Validación automática de datos:** mediante Pydantic, los modelos de datos se definen como clases de Python, y FastAPI valida automáticamente las entradas/salidas, generando documentación interactiva.
- **Documentación integrada:** Swagger UI y ReDoc se generan en /docs y /redoc sin configuración adicional, facilitando el desarrollo frontend y las pruebas.
- **Inyección de dependencias:** sistema limpio y tipado para manejar conexiones a bases de datos, autenticación y otros recursos compartidos.

1.4.2 Ventajas para el proyecto: rendimiento asíncrono y ecosistema Python

La elección de FastAPI para este proyecto está motivada por:

- **Manejo eficiente de concurrencia:** un portal de emergencias puede recibir muchas solicitudes simultáneas. El soporte nativo de operaciones asíncronas permite liberar el hilo de ejecución mientras se esperan tareas de E/S (consultas a base de datos, llamadas a APIs externas), manteniendo alta capacidad de respuesta.
- **Ecosistema de IA/ML:** Python es el lenguaje dominante en inteligencia artificial, aprendizaje automático y procesamiento de datos. Esto facilita la integración directa de modelos de visión artificial (OpenCV, TensorFlow, PyTorch), reconocimiento de audio (librosa, SpeechRecognition) y NLP (Transformers de Hugging Face, spaCy) dentro del backend, sin necesidad de microservicios heterogéneos.
- **Rapidez de prototipado:** la sintaxis concisa y la generación automática de esquemas aceleran el desarrollo, permitiendo iterar rápidamente sobre la lógica de negocio.

1.4.3 Integración nativa con modelos de IA y servicios de datos

El backend con FastAPI se comunicará con PostgreSQL utilizando **SQLAlchemy** (modo asíncrono con asyncio) o **Tortoise-ORM**, ambos compatibles con FastAPI. Los endpoints expondrán servicios para:

- Gestión de usuarios, talleres y solicitudes de auxilio.
- Recepción y almacenamiento de imágenes, audios y textos descriptivos.
- Invocación de pipelines de IA: una vez recibida la evidencia multimedia, FastAPI llamará internamente a los módulos de procesamiento de visión, audio y texto, y devolverá un diagnóstico preliminar junto con la lista de talleres cercanos.
- Integración con pasarelas de pago como Stripe y generación de códigos QR.

1.5 Frontend con Angular

1.1.1 Características de Angular

Angular es un framework de TypeScript mantenido por Google, basado en componentes y orientado a la construcción de **Single Page Applications (SPA)**. Ofrece:

- **Vinculación bidireccional de datos (two-way data binding).**
- **Inyección de dependencias.**
- **Módulos y lazy loading** para optimizar la carga inicial.
- **RxJS** para manejo reactivo de flujos asíncronos (solicitudes HTTP, WebSockets).

1.5.1 Aplicación en el panel web

El frontend Angular servirá como panel de administración para talleres y gestores de la plataforma, permitiendo:

- Gestión de solicitudes de emergencia entrantes.
- Visualización de la ubicación en mapas interactivos.
- Panel de reportes y facturación.
- Configuración de servicios y precios.

1.5.2 Comunicación con el backend FastAPI

Se consumirán las APIs RESTful expuestas por FastAPI mediante el módulo HttpClient de Angular. Se implementará autenticación basada en tokens JWT (por ejemplo, con OAuth2 + python-jose) y un interceptor en Angular para adjuntar las credenciales en cada petición.

1.6 Flutter para Aplicaciones Móviles

1.6.1 Arquitectura y lenguaje Dart

Flutter utiliza **Dart**, un lenguaje optimizado para compilación Ahead-Of-Time (AOT) en código nativo. Su arquitectura declarativa construye la interfaz como un árbol de widgets inmutables, lo que simplifica el estado de la UI y mejora el rendimiento.

1.6.2 Widgets y experiencia de usuario

Gracias a su catálogo de widgets Material Design y Cupertino, la app móvil se adaptará visualmente a Android e iOS, ofreciendo una experiencia nativa. Componentes como Camera, AudioRecorder y GoogleMapsFlutter serán esenciales para la captura de evidencia visual/auditiva y la elección de ubicación.

1.6.3 Integración de pagos y comunicación con APIs

Se implementará la pasarela **Stripe** mediante el paquete flutter_stripe, permitiendo pagos con tarjeta guardados. Para pagos QR, se generará una imagen con los datos de la transacción desde el backend FastAPI y se mostrará en la app del cliente. Todas las interacciones con el servidor se realizarán a través de endpoints REST

documentados automáticamente por FastAPI, facilitando la generación de clientes Dart si se desea.

1.7 Inteligencia Artificial: Visión Artificial, Reconocimiento de Audio y Texto

1.7.1 Visión artificial para diagnóstico de averías

La visión artificial permite a los sistemas extraer información significativa de imágenes y videos. Técnicas como las **redes neuronales convolucionales (CNN)** pueden clasificar el tipo de avería a partir de una fotografía: pinchazo visible, daños en la carrocería, humo en el motor, etc. Mediante detección de objetos (YOLO, SSD) se puede identificar la presencia de neumáticos dañados, fugas de líquido o piezas sueltas, proporcionando un diagnóstico preliminar antes de la llegada del mecánico.

1.7.2 Reconocimiento de audio y procesamiento de lenguaje natural

- **Audio:** un modelo de clasificación de sonidos (p.ej., utilizando espectrogramas y CNN o arquitecturas como Wav2Vec) puede identificar ruidos característicos (motor de arranque fallando, golpeteo metálico, sirenas de emergencia) a partir de una grabación del usuario.
- **Texto:** el procesamiento de lenguaje natural (NLP) analiza la descripción escrita por el cliente (por ejemplo, “no arranca y hace un clic”) mediante modelos pre-entrenados (BERT, GPT, FastText) para extraer la intención y la gravedad del problema.

1.7.3 Librerías y servicios (públicos y locales)

- **Librerías locales:** TensorFlow Lite y PyTorch Mobile permiten ejecutar modelos de IA en el propio dispositivo móvil (análisis offline de imágenes o audio), reduciendo latencia y consumo de datos. En el backend con FastAPI, se pueden servir modelos almacenados en formatos como ONNX o PyTorch, integrándolos directamente en los endpoints.
- **Servicios cloud públicos:** Azure Cognitive Services, Google Cloud Vision / Video Intelligence, AWS Rekognition y Amazon Transcribe ofrecen APIs listas para usar en tareas de clasificación visual y transcripción de audio, con alta precisión sin entrenar modelos propios. Esto acelera el desarrollo y puede ser una capa complementaria antes de invertir en modelos personalizados.

1.7.4 Modelos supervisados y no supervisados

- **Supervisados:** requieren datos etiquetados (imagen del pinchazo → “pinchazo”; audio de motor → “motor”). Son ideales para clasificación precisa del problema. Se pueden entrenar con conjuntos de datos específicos del taller o utilizando transfer learning a partir de modelos pre-entrenados.
- **No supervisados:** detectan patrones sin etiquetas, útiles para agrupar tipos de averías emergentes o segmentar clientes según su comportamiento. Técnicas como clustering (K-means, DBSCAN) pueden ayudar a refinar la oferta de servicios.

1.7.5 Aplicación en el proyecto: triaje automático del problema

La IA actuará como un **triaje automático**:

1. El usuario sube una foto, un audio y/o una descripción textual de la avería.
2. Un pipeline de IA, implementado dentro del backend FastAPI o mediante servicios en la nube, analiza las entradas y emite un diagnóstico: “pinchazo en neumático delantero derecho”, “fallo de batería”, “accidente con daños estructurales”.
3. El sistema sugiere el tipo de servicio y el taller especializado más cercano, agilizando la atención.
4. El mecánico recibe la solicitud con información precisa y herramientas necesarias, reduciendo el tiempo de resolución y mejorando la seguridad vial.

2 PUDS

2.1 FT:Captura de Requisitos

2.1.1 Identificar Actores y Casos de Uso

2.1.1.1 Identificar Actores

ID	ACTOR	DESCRIPCIÓN
A1	Administrador	Responsable de supervisar la operación general del sistema, consultar todos los incidentes y ejecutar procesos de asignación cuando sea necesario.
A2	Cliente	Usuario que registra su cuenta, gestiona sus vehículos, reporta emergencias, adjunta evidencias, consulta el estado de sus incidentes y realiza pagos.
A3	Taller	Usuario encargado de registrar y administrar su taller, gestionar técnicos, revisar incidentes disponibles, aceptar solicitudes y actualizar el estado del servicio.

2.1.1.2 Identificar Casos de Uso

ID	NOMBRE
CU-01	Gestionar cuenta de usuario
CU-02	Iniciar sesión
CU-03	Cerrar sesión
CU-04	Gestionar perfil Cliente
CU-05	Gestionar vehículos
CU-06	Reportar emergencia vehicular
CU-07	Gestionar evidencias del incidente
CU-08	Consultar incidentes
CU-09	Consultar detalle e historial del incidente
CU-10	Gestionar notificaciones
CU-11	Gestionar pagos
CU-12	Gestionar taller
CU-13	Gestionar técnicos
CU-14	Gestionar incidentes asignados

CU-15	Asignar taller automáticamente
CU-16	Procesar incidente con IA

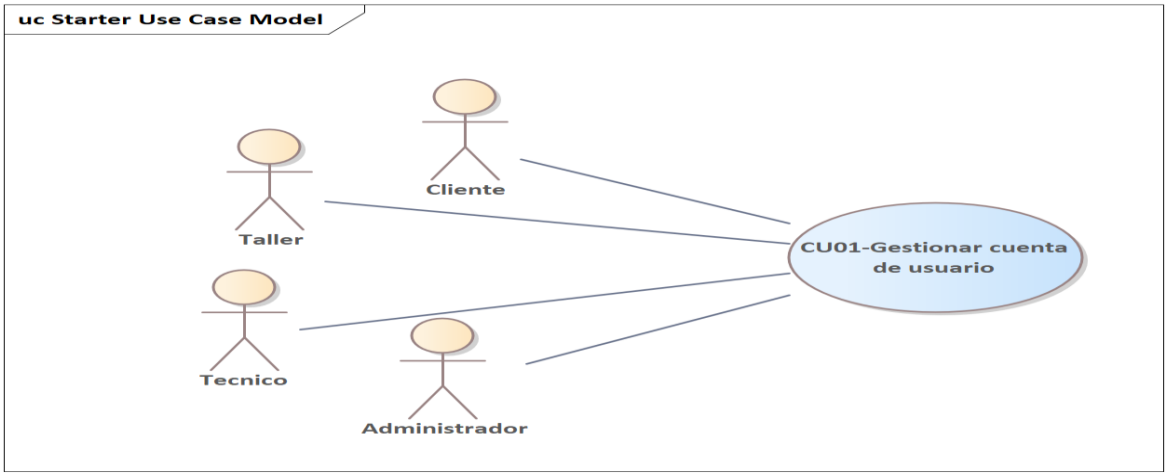
2.1.2 Priorizar Casos de Uso

ID	NOMBRE	ACTOR	PRIORIDAD
CU-01	Gestionar cuenta de usuario	Cliente	ALTA
CU 2	Iniciar sesión	Administrador, Cliente, Taller,Tecnico	ALTA
CU 3	Cerrar session	Administrador, Cliente, Taller	MEDIA
CU 4	Gestionar Perfil	Cliente, Taller,Tecnico	MEDIA
CU 5	Gestionar vehículos	Cliente	ALTA
CU 6	Reportar emergencia vehicular	Cliente	ALTA
CU 7	Gestionar evidencias del incidente	Cliente	ALTA
CU 8	Consultar incidentes	Administrador, Cliente, Taller,Tecnico	ALTA
CU 9	Consultar detalle e historial del incidente	Administrador, Cliente, Taller,Tecnico	ALTA

CU 10	Gestionar notificaciones	Cliente, Taller,Tecnico	MEDIA
CU 11	Gestionar pagos	Cliente	ALTA
CU 12	Gestionar taller	Taller	ALTA
CU 13	Gestionar técnicos	Taller	MEDIA
CU 14	Gestionar incidentes asignados	Taller,Tecnico	ALTA
CU 15	Asignar taller automáticamente	Administrador	ALTA
CU 16	Procesar incidente con IA	Sistema Inteligente	MEDIA
CU 17	Consultar talleres cercanos	Cliente, Administrador	MEDIA
CU 18	Consultar talleres candidatos para asignación	Administrador	MEDIA

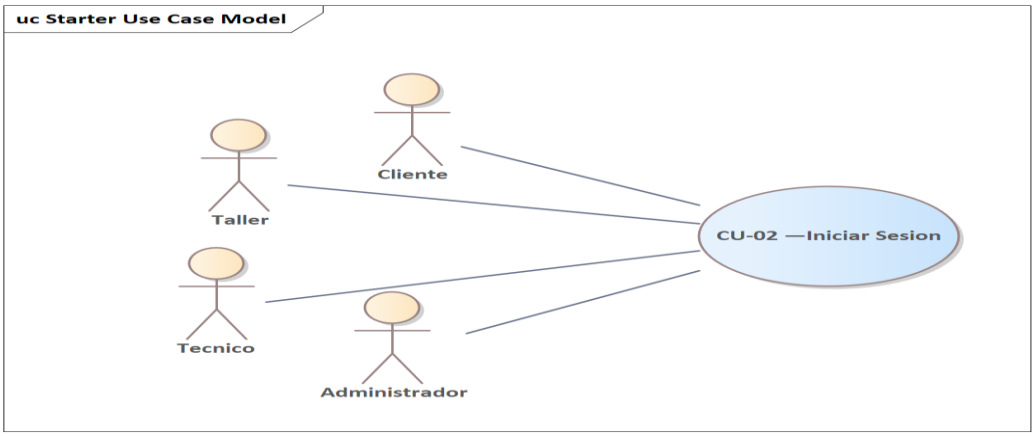
2.1.3 Especificar Casos de Uso

CU01-Gestionar cuenta de usuario



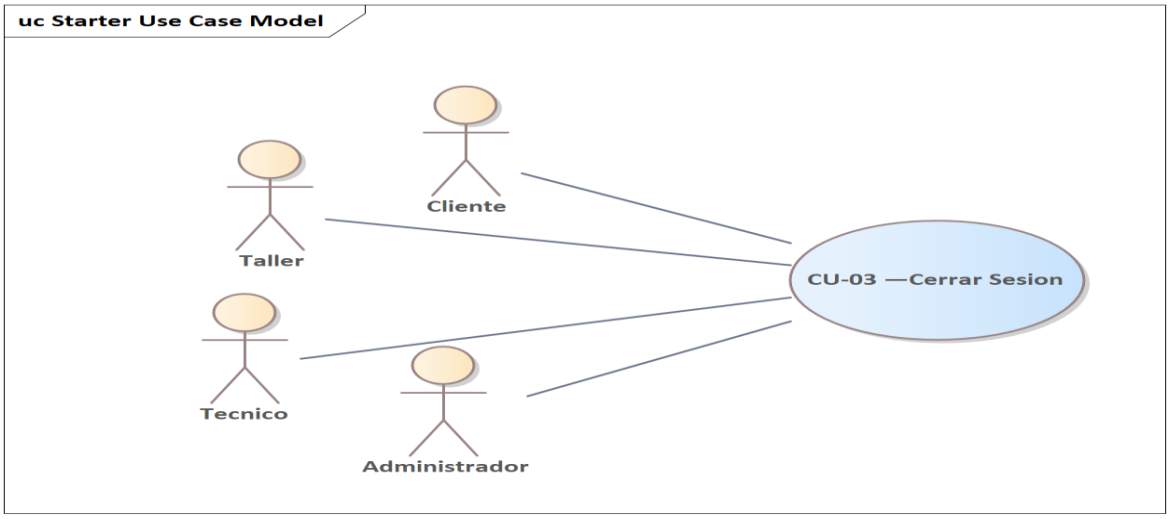
Propósito	Permitir que los usuarios (Administrador, Funcionario o Cliente) accedan al sistema mediante la validación de sus credenciales.
Resumen	El usuario ingresa su email y contraseña. El sistema valida los datos contra la base de datos y, si son correctos, genera un token JWT para manejar la sesión y los permisos según el rol.
Actores	Cliente,Taller,Tecnico,Administrador
Actor Iniciador	Cliente,Taller,Tecnico,Administrador
Precondicion	El usuario debe estar registrado en el sistema.
Postcondicion	El usuario accede al sistema con una sesión activa
Flujo Principal	1. El usuario accede a la interfaz de inicio de sesión. 2. El usuario ingresa su correo electrónico y contraseña. 3. El sistema valida las credenciales en la base de datos. 4. El sistema genera un token de acceso (JWT) con el rol del usuario. 5. El sistema redirige al usuario al dashboard correspondiente a su nivel de acceso.
Excepcion	**Credenciales inválidas:** El sistema deniega el acceso y muestra un mensaje de error indicando que el usuario o la contraseña son incorrectos.

CU-02 —Iniciar Sesion



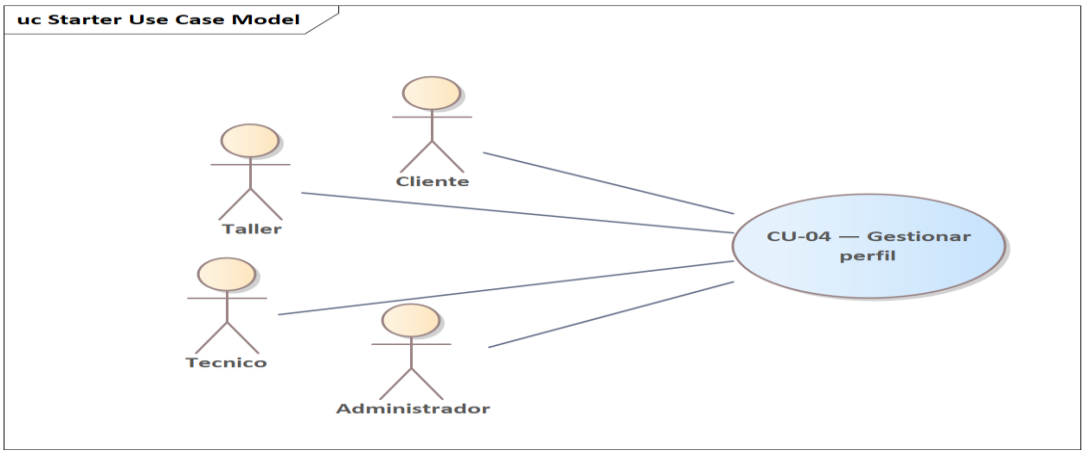
Propósito	Permitir que los usuarios registrados accedan al sistema mediante la validación de sus credenciales.
Resumen	El usuario ingresa su correo y contraseña. El sistema valida los datos contra la base de datos y, si son correctos, genera un token JWT para manejar la sesión y los permisos según el rol del usuario.
Actores	Cliente, Taller, Administrador,Tecnico
Actor Iniciador	Cliente, Taller, Administrador,Tecnico
Precondicion	El usuario debe estar registrado en el sistema.
Postcondicion	El usuario accede al sistema con una sesión activa y un token JWT válido.
Flujo Principal	1. El usuario accede a la pantalla de inicio de sesión Ingresa su nombre completo, correo electrónico, número de teléfono y contraseña. 2. Ingresa su correo electrónico y contraseña.El sistema comprueba que el correo no esté en uso. 3. El sistema valida las credenciales contra la base de datos 4. El sistema genera un token de acceso JWT con el rol del usuario.
Excepcion	El sistema redirige al usuario a la pantalla principal correspondiente a su nivel de acceso. Excepción Credenciales inválidas: el sistema deniega el acceso y muestra un mensaje indicando que el correo o la contraseña son incorrectos. El usuario puede intentarlo nuevamente.

CU-03 —Cerrar Sesion



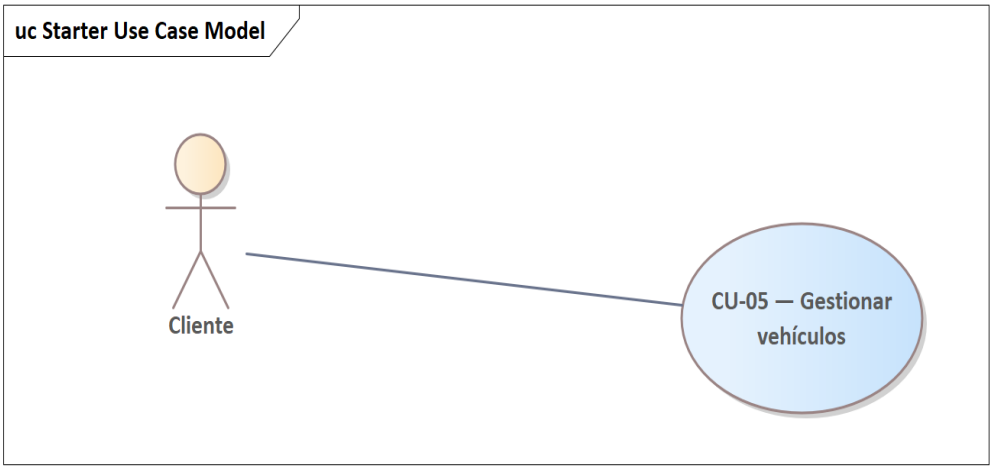
Propósito	Permitir que el usuario finalice su sesión activa y salga del sistema de forma segura.
Resumen	El usuario selecciona la opción de cerrar sesión desde el menú de la aplicación. El sistema elimina el token JWT almacenado localmente y redirige al usuario a la pantalla de inicio de sesión, sin posibilidad de acceder nuevamente hasta que vuelva a autenticarse.
Actores	Cliente, Taller, Administrador,Tecnico
Actor Iniciador	Cliiente, Taller, Administrador,Tecnico
Precondicion	El usuario debe tener una sesión activa en la aplicación.
Postcondicion	El token de sesión es eliminado y el usuario pierde el acceso hasta volver a iniciar sesión.
Flujo Principal	1. El usuario accede al menú de opciones desde la pantalla principal. 2. Selecciona la opción "Cerrar sesión" 3. El sistema elimina el token JWT y los datos de sesión almacenados en el dispositivo. 4. El sistema redirige al usuario a la pantalla de inicio de sesión.
Excepcion	Si no existe sesión activa en el dispositivo, el sistema redirige directamente a la pantalla de inicio de sesión sin ejecutar ninguna acción adicional.

CU-04 — Gestionar perfil



Propósito	Permitir que el usuario consulte y actualice sus datos personales registrados en el sistema.
Resumen	El usuario accede a su perfil para visualizar su información actual (nombre, teléfono). Si desea realizar cambios, modifica los campos necesarios y el sistema actualiza los datos en la base de datos, devolviendo el perfil actualizado.
Actores	Cliente, Taller,Tecnico
Actor Iniciador	Cliente, Taller,Tecnico
Precondicion	El usuario debe tener una sesión activa.
Postcondicion	Los datos del perfil quedan actualizados en el sistema.
Flujo Principal	1. El usuario accede a la sección de perfil.. 2. El sistema muestra los datos actuales del usuario. 3. El usuario modifica los campos que desea actualizar (nombre o teléfono).. 4. El sistema valida que los datos ingresados sean correctos. 5. El sistema guarda los cambios y confirma la actualización al usuario
Excepcion	Datos inválidos: el sistema informa al usuario qué campos son incorrectos y solicita corrección antes de guardar. Sesión expirada: el sistema solicita al usuario que vuelva a iniciar sesión

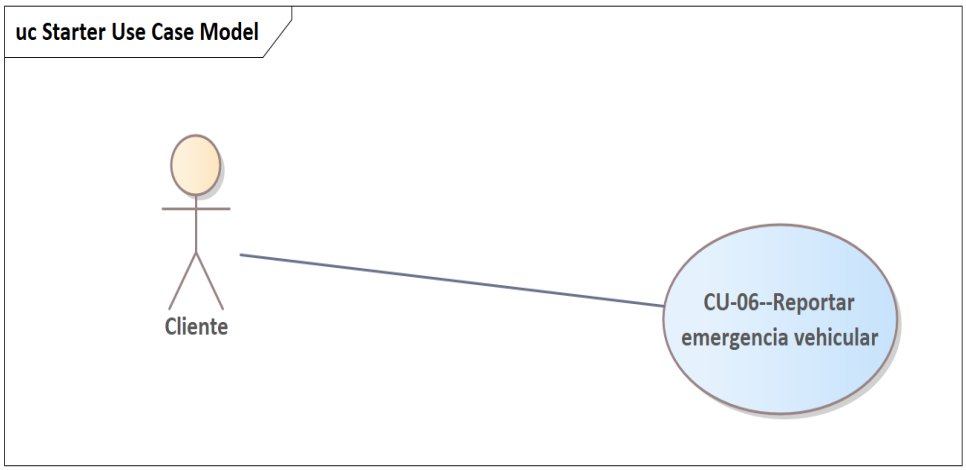
CU-05 — Gestionar vehículos



Propósito	Permitir que el cliente administre los vehículos asociados a su cuenta para poder reportar emergencias con ellos.
Resumen	El cliente puede registrar nuevos vehículos indicando marca, modelo, año, placa y color. También puede consultar la lista de sus vehículos, actualizar sus datos o eliminarlos. El sistema garantiza que no existan dos vehículos con la misma placa.
Actores	Cliente
Actor Iniciador	Cliente
Precondicion	El cliente debe tener una sesión activa.
Postcondicion	El vehículo queda registrado, actualizado o eliminado según la acción realizada.
Flujo Principal	1. El cliente accede a la sección "Mis vehículos" 2. El sistema muestra la lista de vehículos registrados. 3. El cliente selecciona la acción a realizar: registrar, editar o eliminar un vehículo 4. Si registra: ingresa los datos del vehículo y el sistema verifica que la placa no esté en uso, luego guarda el nuevo vehículo 5. Si edita: modifica los datos deseados y el sistema guarda los cambios 6. Si elimina: confirma la acción y el sistema elimina el vehículo de la cuenta.

Excepcion	Placa duplicada: el sistema informa que ya existe un vehículo registrado con esa placa. Vehículo no encontrado al editar o eliminar: el sistema notifica que el vehículo no existe o no pertenece al usuario.

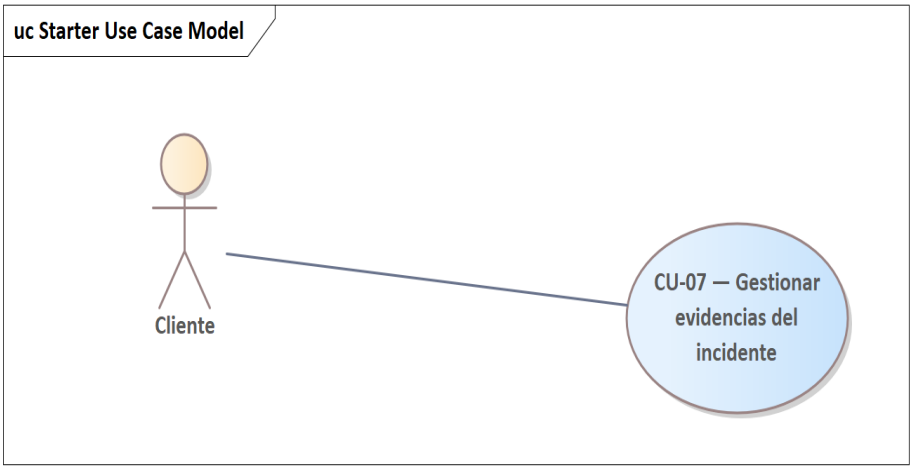
CU-06--Reportar emergencia vehicular



Propósito	Permitir que el cliente solicite asistencia vehicular de emergencia indicando su vehículo, ubicación y descripción del problema.
Resumen	El cliente selecciona el vehículo afectado, obtiene su ubicación actual mediante GPS y describe el problema opcionalmente. El sistema registra el incidente con estado pendiente, almacena la descripción como evidencia de texto y lo deja disponible para la asignación de un taller.
Actores	Cliente
Actor Iniciador	Cliente
Precondicion	l cliente debe tener sesión activa y al menos un vehículo registrado.
Postcondicion	El incidente queda registrado con estado "pendiente" y disponible para ser atendido por un taller.

Flujo Principal	1. El cliente presiona el botón "EMERGENCIA" en la pantalla principal 2. Selecciona el vehículo que presenta el problema. 3. El sistema obtiene la ubicación actual del dispositivo mediante GPS. 4. El cliente describe brevemente el problema (opcional 5. El cliente confirma el reporte. 6. El sistema registra el incidente con estado pendiente y lo notifica como disponible para atención.
Excepcion	Sin vehículos registrados: el sistema indica que el cliente debe registrar al menos un vehículo antes de reportar una emergencia. Sin permisos de ubicación: el sistema utiliza una ubicación predeterminada e informa al cliente que active los permisos GPS para mayor precisión.

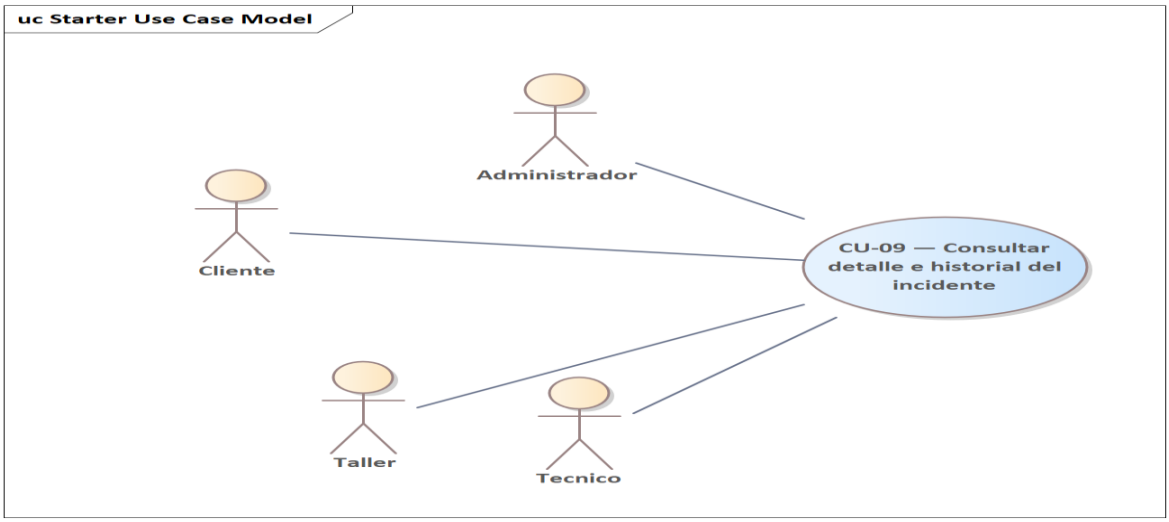
CU-07 — Gestionar evidencias del incidente



Propósito	Permitir que el cliente adjunte fotografías o grabaciones de voz como evidencia del problema vehicular, para facilitar el diagnóstico por parte del sistema de inteligencia artificial.
Resumen	El cliente, durante o después de reportar la emergencia, puede adjuntar imágenes tomadas desde la cámara o galería, y grabaciones de audio describiendo el problema. El sistema almacena los archivos, los asocia al incidente correspondiente y los procesa automáticamente con inteligencia artificial para generar un diagnóstico.
Actores	Cliente

Actor Iniciador	Cliente
Precondicion	El incidente debe estar registrado en el sistema y pertenecer al cliente autenticado.
Postcondicion	Las evidencias quedan registradas y asociadas al incidente. El sistema inicia el análisis automático con inteligencia artificial.
Flujo Principal	1. El cliente accede al incidente activo.. 2. Selecciona la opción de adjuntar evidencia. 3. Elige el tipo de evidencia: imagen (desde cámara o galería) o audio. 4. El sistema recibe el archivo, lo almacena de forma segura y lo vincula al incidente. 5. El sistema procesa automáticamente la evidencia mediante inteligencia artificial 6. El diagnóstico generado queda disponible en el detalle del incidente.
Excepcion	El incidente no pertenece al cliente: el sistema deniega el acceso a esa evidencia. Fallo al subir el archivo: el sistema informa al cliente que hubo un problema y solicita intentarlo nuevamente.

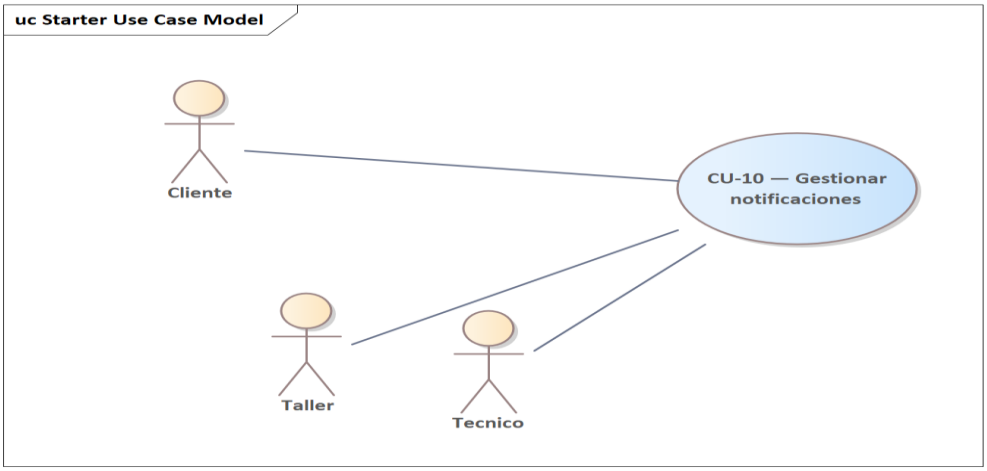
CU-09 — Consultar detalle e historial del incidente



Propósito	Permitir que los usuarios consulten la información completa de un incidente, incluyendo su diagnóstico de inteligencia artificial, evidencias adjuntas y el historial de cambios de estado

Resumen	El usuario selecciona un incidente de la lista. El sistema muestra todos sus datos: categoría, prioridad, estado actual, resumen generado por la IA, diagnóstico, ubicación, costo final, tiempo estimado de llegada, evidencias adjuntas y el historial completo de cambios de estado con fecha, responsable y notas.
Actores	Cliente, Taller, Administrador,Tecnico
Actor Iniciador	Cliente, Taller, Administrador,Tecnico
Precondicion	El incidente debe existir en el sistema. El Cliente solo puede ver sus propios incidentes
Postcondicion	El sistema muestra la información detallada e historial del incidente seleccionado.
Flujo Principal	1. El usuario accede a la lista de incidentes 2. Selecciona el incidente que desea consulta 3. El sistema recupera toda la información del incidente: datos generales, diagnóstico de la IA, evidencias adjuntas e historial de estados. 4. El sistema presenta la información de forma organizada al usuario.
Excepcion	El cliente intenta acceder a un incidente que no le pertenece: el sistema deniega el acceso y muestra un mensaje de permiso insuficiente. El incidente no existe: el sistema informa que no se encontró la información solicitada..

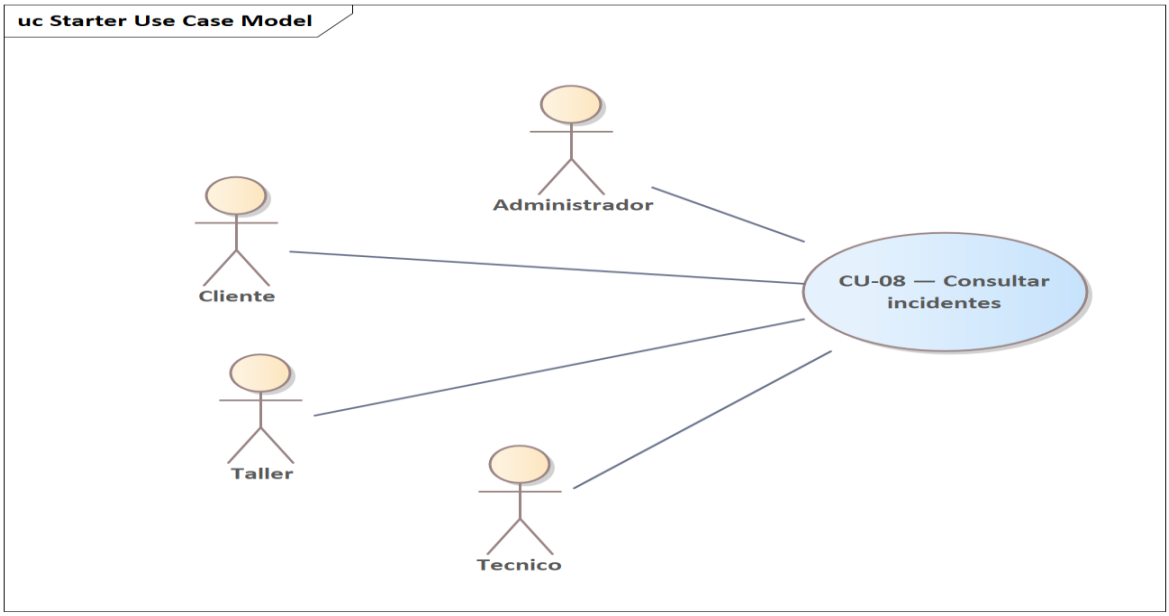
CU-10 — Gestionar notificaciones



Propósito	Permitir que el usuario consulte las notificaciones que el sistema ha generado sobre sus incidentes y gestione su estado de lectura.
Resumen	El sistema genera notificaciones automáticas ante eventos como la asignación de un taller, cambios de estado del servicio o confirmación de pago. El usuario puede consultar el listado de sus notificaciones, ver cuántas no ha leído, marcar una notificación individual como leída o marcarlas todas como leídas a la vez.
Actores	Cliente, Taller,Tecnico
Actor Iniciador	Cliente, Taller,Tecnico
Precondicion	El usuario debe tener una sesión activa.
Postcondicion	Las notificaciones seleccionadas quedan marcadas como leídas en el sistema.
Flujo Principal	1. El usuario accede a la sección de notificaciones o visualiza el contador de no leídas en la pantalla principal 2. El sistema muestra las últimas 50 notificaciones ordenadas por fecha, indicando cuáles han sido leídas. 3. . El usuario puede seleccionar una notificación para marcarla como leída 4. Alternativamente, puede seleccionar la opción "marcar todas como leídas" para actualizar todas de una vez 5. El sistema actualiza el estado de lectura y refleja el cambio en el contador de la pantalla principal.

Excepcion	Sin notificaciones: el sistema informa que no hay notificaciones disponibles. Notificación no encontrada o no perteneciente al usuario: el sistema ignora la acción sin mostrar un error visible.

CU-08 — Consultar incidentes



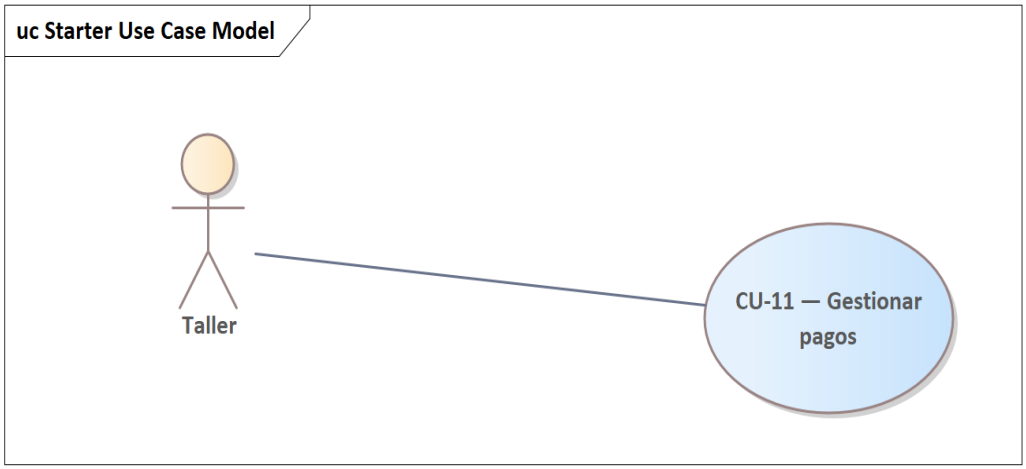
Propósito	Permitir que los usuarios visualicen la lista de incidentes registrados según su rol en el sistema
Resumen	El cliente visualiza únicamente sus propios incidentes. El taller ve los incidentes que le han sido asignados más los disponibles pendientes de atención. El administrador tiene acceso a todos los incidentes del sistema. En todos los casos se muestra la información resumida de cada incidente ordenada por fecha..
Actores	Cliente, Taller, Administrador,Tecnico
Actor Iniciador	Cliente, Taller, Administrador,Tecnico
Precondicion	El usuario debe tener una sesión activa.
Postcondicion	El sistema muestra la lista de incidentes correspondiente al rol del usuario.

Flujo Principal	1. El usuario accede a la pantalla principal o sección de incidentes. 2. El sistema identifica el rol del usuario autenticado 3. Si es Cliente: muestra sus propios incidentes ordenados por fecha 4. Si es Taller: muestra los incidentes asignados a su taller y los pendientes disponibles. 5. Si es Administrador: muestra todos los incidentes del sistema.
Excepcion	Sin incidentes disponibles: el sistema muestra un mensaje indicando que no hay solicitudes activas. Sesión expirada: el sistema redirige al usuario a la pantalla de inicio de sesión.

CU-11 — Gestionar pagos	
Propósito	Permitir que el cliente registre el pago del servicio recibido y consulte su historial de pagos anteriores.
Resumen	Una vez completado el servicio, el cliente puede registrar el pago indicando el monto y el método de pago utilizado. El sistema registra la transacción, actualiza el costo final del incidente y genera una notificación de confirmación. El cliente también puede consultar todos sus pagos anteriores.
Actores	Cliente
Actor Iniciador	Cliente
Precondicion	El cliente debe tener sesión activa y el incidente asociado debe pertenecerle. No debe existir un pago previo registrado para ese mismo incidente.
Postcondicion	El pago queda registrado, el costo final del incidente se actualiza y el cliente recibe una notificación de confirmación.

CU-11 — Gestionar pagos	
Flujo Principal	1. El cliente accede al detalle del incidente atendido. 2. Selecciona la opción de realizar el pago. 3. Ingresa el monto del servicio y el método de pago. 4. El sistema calcula automáticamente la comisión de la plataforma (10% del monto).. 5. El sistema registra el pago como completado y actualiza los datos del incidente. 6. El sistema envía una notificación al cliente confirmando el pago realizado 7. El cliente puede consultar este y todos sus pagos anteriores desde la sección de historial.
Excepcion	Pago duplicado: el sistema informa que ya existe un pago registrado para ese incidente. Incidente no encontrado o no perteneciente al cliente: el sistema deniega la operación.

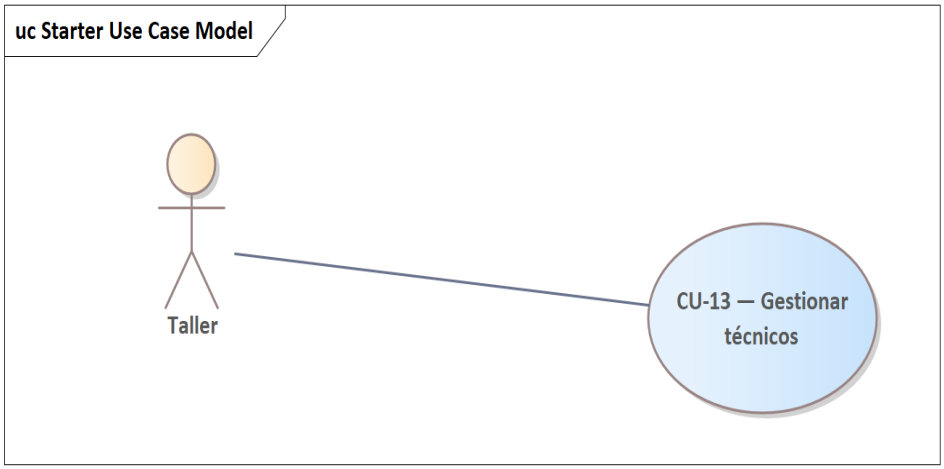
CU-12 — Gestionar taller



Propósito	Permitir que un usuario con rol Taller registre y administre la información de su taller dentro del sistema para poder recibir y atender solicitudes de emergencia.
Resumen	El usuario con rol Taller puede registrar su taller indicando nombre, dirección, ubicación, capacidad, servicios que ofrece y calificación inicial. Una vez registrado, puede consultar los datos de su taller y actualizarlos cuando sea necesario. Cada usuario de tipo Taller puede gestionar únicamente un taller.
Actores	Taller

Actor Iniciador	Taller
Precondicion	El usuario debe tener sesión activa con rol Taller. Para el registro, no debe tener un taller previamente creado
Postcondicion	El taller queda registrado o actualizado en el sistema y disponible para recibir asignaciones de incidentes.
Flujo Principal	1. El usuario con rol Taller accede a la sección de gestión de su taller 2. Si no tiene taller registrado: ingresa los datos del taller (nombre, dirección, coordenadas, capacidad y servicios) y el sistema lo registra 3. Si ya tiene taller registrado: el sistema muestra la información actual. 4. El usuario puede modificar los datos que desee actualizar 5. El sistema guarda los cambios y confirma la actualización.
Excepcion	El usuario intenta registrar un segundo taller: el sistema informa que ya tiene uno registrado y no permite crear otro. Un usuario con rol diferente a Taller intenta registrar un taller: el sistema deniega la acción.

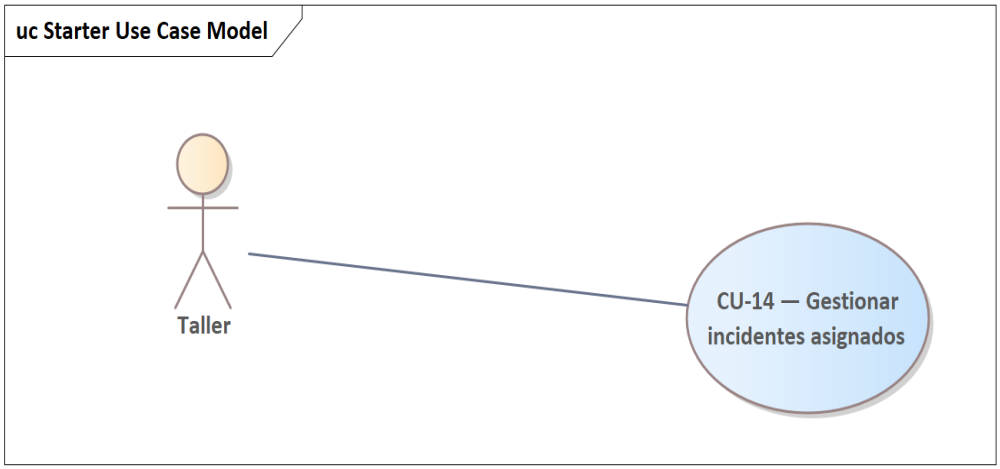
CU-13 — Gestionar técnicos



Propósito	Permitir que el Taller administre los técnicos que trabajan bajo su cargo, para que el sistema pueda asignarlos a incidentes de forma eficiente
Resumen	El usuario con rol Taller puede registrar nuevos técnicos indicando nombre, especialidades y ubicación. También puede consultar la lista de técnicos activos de su taller, actualizar sus datos o eliminarlos cuando ya no formen parte del equipo. Estos técnicos son considerados por el motor de asignación automática al momento de atender emergencias.
Actores	Taller
Actor Iniciador	Taller
Precondicion	El usuario debe tener sesión activa con rol Taller y tener un taller registrado en el sistema.
Postcondicion	El técnico queda registrado, actualizado o eliminado según la acción realizada.
Flujo Principal	1. El Taller accede a la sección de gestión de técnicos. 2. El sistema muestra la lista de técnicos registrados en su taller. 3. El Taller selecciona la acción a realizar: registrar, editar o eliminar un técnico.. 4. Si registra: ingresa el nombre y especialidades del técnico, y el sistema lo vincula al taller 5. . i edita: modifica los datos necesarios y el sistema guarda los cambios

	6. Si elimina: confirma la acción y el sistema elimina al técnico del taller.
Excepcion	da los cambios. 6. Si elimina: confirma la acción y el sistema elimina al técnico del taller. Excepción El taller no tiene un taller registrado: el sistema informa que primero debe completar el registro del taller. Técnico no encontrado o no perteneciente al taller: el sistema rechaza la operación.

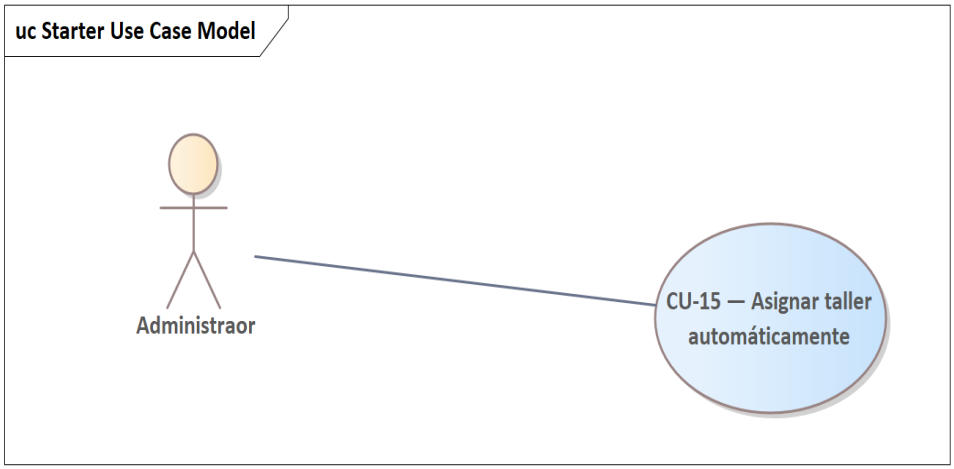
CU-14 — Gestionar incidentes asignados



Propósito	Permitir que el Taller administre los incidentes que recibe, pudiendo aceptarlos, rechazarlos y actualizar su estado a lo largo de la atención.
Resumen	El Taller visualiza los incidentes pendientes disponibles y los que ya tiene asignados. Puede aceptar un incidente para tomarlo a su cargo, asignarle un técnico y comprometerse con la atención. Si no puede atenderlo, puede rechazarlo para dejarlo disponible nuevamente. Una vez en atención, puede actualizar el estado del servicio, registrar el costo final y el tiempo estimado de llegada. Cada cambio de estado genera una notificación automática al cliente.
Actores	Taller,Tecnico
Actor Iniciador	Taller,Tecnico

Precondicion	El usuario debe tener sesión activa con rol Taller y tener un taller registrado
Postcondicion	El incidente queda aceptado, rechazado o actualizado según la acción realizada. El cliente recibe una notificación con cada cambio relevante.
Flujo Principal	1. El Taller accede a la lista de incidentes disponibles y asignados. 2. Selecciona un incidente pendiente para revisarlo. 3. Si decide aceptarlo: el sistema asigna el taller y el técnico seleccionado al incidente, cambia su estado a "asignado" y notifica al cliente. 4. Si decide rechazarlo: el sistema devuelve el incidente al estado pendiente para que otro taller pueda tomarlo. 5. Durante la atención, el Taller actualiza el estado del servicio (en progreso, completado, cancelado) según corresponda. 6. El Taller puede registrar el costo final y el tiempo estimado de llegada del técnico. 7. Cada cambio de estado genera una notificación automática al cliente.
Excepcion	El incidente ya fue tomado por otro taller: el sistema informa que ya no está disponible. El Taller intenta rechazar un incidente que no le fue asignado: el sistema deniega la operación.

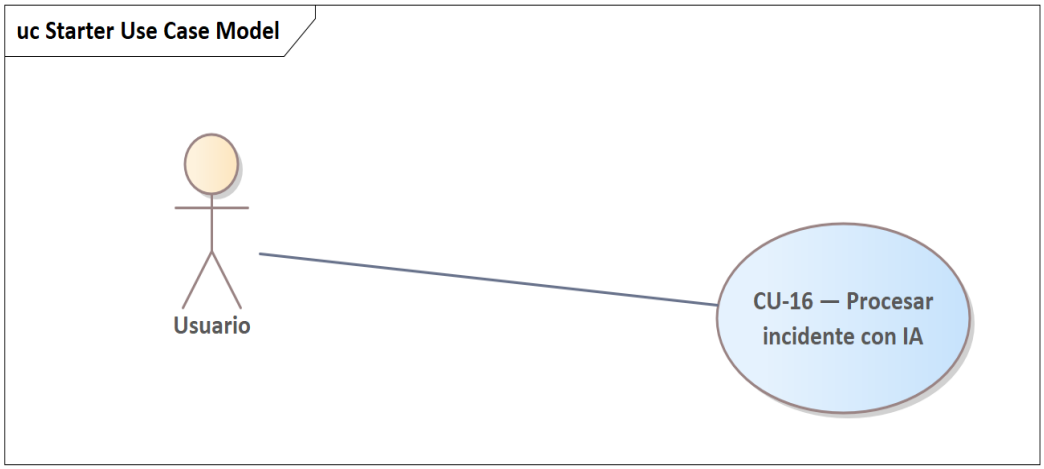
CU-15 — Asignar taller automáticamente



Propósito	Permitir que el sistema asigne de forma automática el taller y técnico más adecuados a un incidente pendiente, sin intervención manual.
Resumen	Cuando un incidente no ha sido atendido manualmente, el sistema ejecuta su motor de asignación inteligente. Evalúa todos los talleres disponibles considerando la distancia al lugar del incidente, los servicios que ofrece cada taller, la disponibilidad de técnicos con las especialidades requeridas, la calificación del taller y la prioridad del caso. Selecciona el mejor candidato, asigna el técnico más cercano y estima el tiempo de llegada. Tanto el cliente como el taller asignado reciben una notificación.
Actores	Administrador
Actor Iniciador	Administrador
Precondicion	El incidente debe existir en el sistema con estado pendiente. Deben existir talleres disponibles con técnicos activos.
Postcondicion	El incidente queda asignado al taller y técnico seleccionados. Ambas partes reciben una notificación de la asignación.
Flujo Principal	1. El sistema detecta un incidente en estado pendiente. 2. El motor de asignación consulta todos los talleres disponibles. 3. Filtra los que ofrecen el tipo de servicio requerido y cuentan con técnicos disponibles. 4. Calcula una puntuación para cada candidato considerando distancia, calificación, capacidad, número de técnicos y prioridad del caso.

	5. Selecciona el taller con mejor puntuación y el técnico más cercano al lugar del incidente 6. Estima el tiempo de llegada en minutos según la distancia 7. l sistema asigna el taller y el técnico al incidente y actualiza su estado a "asignado" 8. Se envía una notificación al cliente con el nombre del taller y el tiempo estimado, y al taller con los datos del nuevo incidente asignado.
Excepcion	No hay talleres disponibles que cumplan los criterios: el sistema informa que no fue posible realizar la asignación automática y el incidente permanece pendiente.

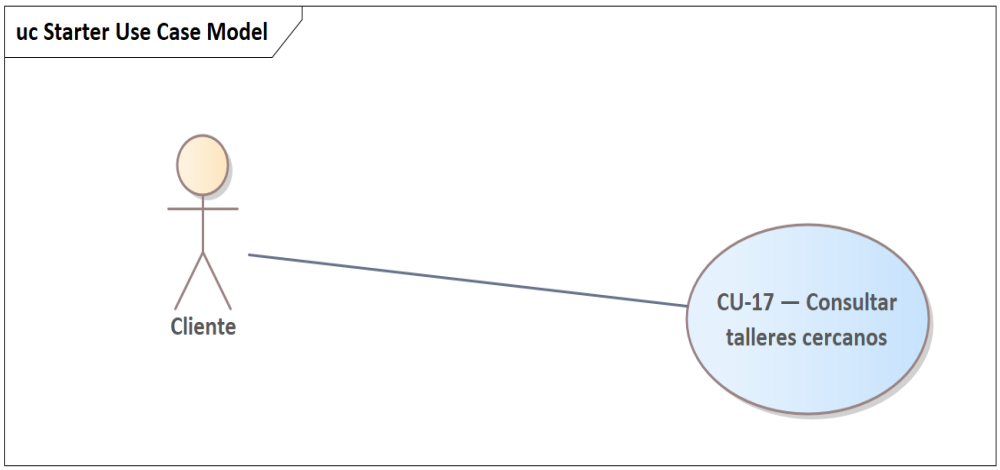
CU-16 — Procesar incidente con IA



Propósito	Permitir que el sistema analice automáticamente las evidencias adjuntas de un incidente para clasificarlo, determinar su prioridad y generar un diagnóstico preliminar.
Resumen	Cuando el cliente adjunta imágenes o grabaciones de audio, el sistema las procesa en segundo plano mediante inteligencia artificial. Las imágenes son analizadas visualmente para identificar daños en el vehículo. Los audios son transcritos y se extraen los puntos clave de la descripción del problema. Con toda la información recopilada, el sistema clasifica el tipo de incidente (batería, llanta, choque, motor, llaves u otro), determina su nivel de prioridad y genera un resumen y diagnóstico preliminar que quedan disponibles en el detalle del incidente.
Actores	Usuario
Actor Iniciador	Usuario (se activa automáticamente al subir una evidencia)
Precondicion	El incidente debe existir y tener al menos una evidencia adjunta (imagen o audio).
Postcondicion	El incidente queda clasificado con categoría, nivel de prioridad, resumen y diagnóstico generados por la IA.
Flujo Principal	1. El sistema detecta que se ha adjuntado una nueva evidencia a un incidente. 2. Si es audio: transcribe el contenido de voz a texto y extrae la información clave del problema descrito. 3. Si es imagen: analiza visualmente el estado del vehículo para identificar daños o condiciones relevantes..

	4. Con todas las evidencias procesadas, el sistema envía la información al módulo clasificador de incidentes. 5. El clasificador determina la categoría del problema, el nivel de prioridad y genera un resumen y diagnóstico preliminar 6. El sistema actualiza el incidente con los resultados y los deja disponibles para el cliente y el taller asignado.
Excepcion	Fallo en el análisis de la evidencia: el sistema registra el error en el campo de análisis y continúa con las demás evidencias sin interrumpir el proceso. No se encontraron evidencias procesables: el sistema no realiza cambios en la clasificación del incidente

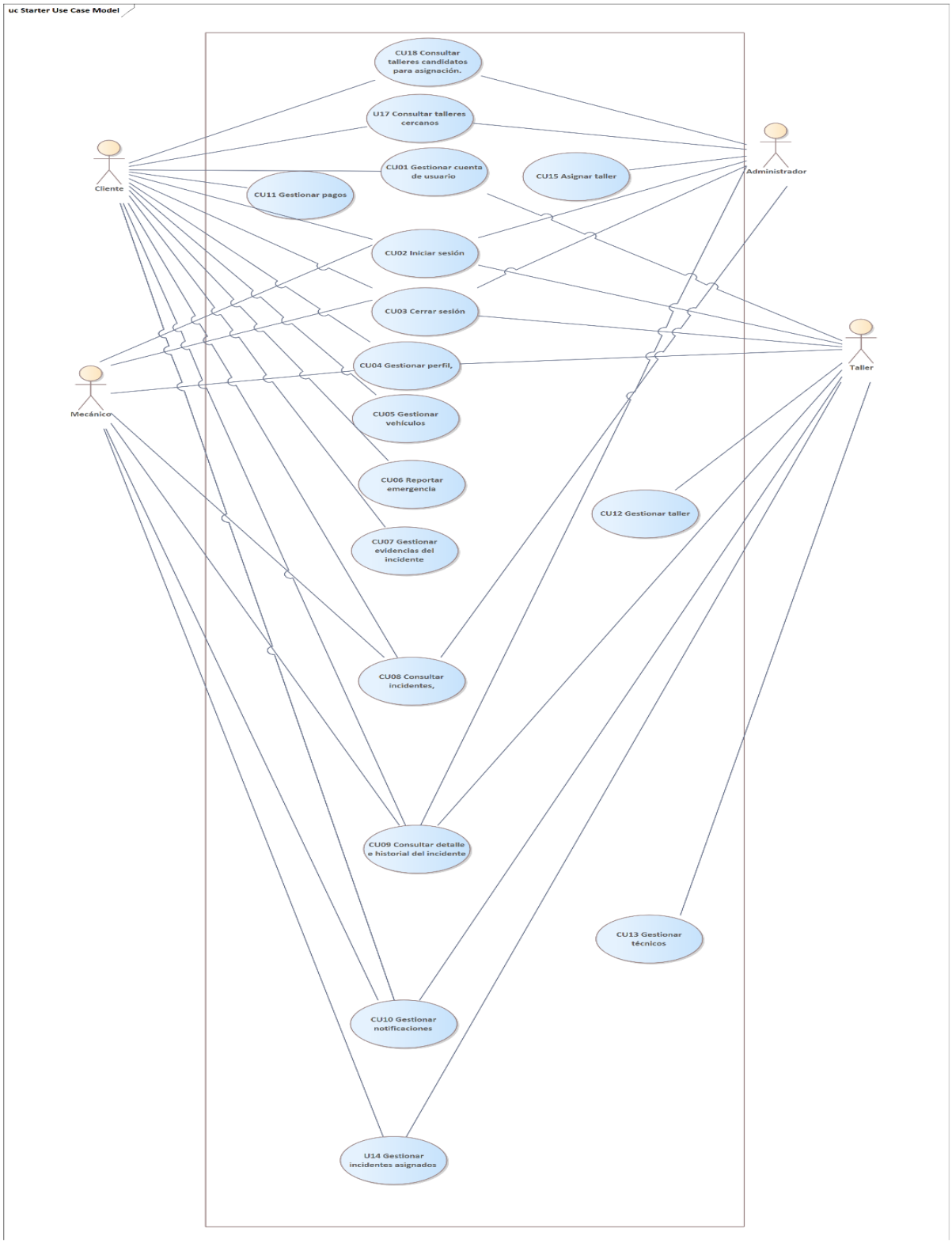
CU-17 — Consultar talleres cercanos



Propósito	Permitir que el sistema analice automáticamente las evidencias adjuntas de un incidente para clasificarlo, determinar su prioridad y generar un diagnóstico preliminar.
Resumen	A partir de unas coordenadas de referencia y un radio de búsqueda (por defecto 10 km), el sistema consulta todos los talleres activos y disponibles, calcula la distancia de cada uno respecto al punto indicado y devuelve la lista ordenada de los más cercanos al más lejano.
Actores	Cliente
Actor Iniciador	Cliente
Precondicion	Deben existir talleres registrados y disponibles en el sistema. Se debe contar con coordenadas de referencia (latitud y longitud).

Postcondicion	El sistema devuelve la lista de talleres disponibles ordenados por proximidad a la ubicación indicada.
Flujo Principal	1. El usuario o el sistema proporciona las coordenadas de referencia (latitud y longitud) y opcionalmente el radio de búsqueda 2. El sistema consulta todos los talleres marcados como disponibles. 3. . Calcula la distancia entre cada taller y el punto de referencia. 4. Filtra los talleres que se encuentran dentro del radio establecido 5. Devuelve la lista ordenada del taller más cercano al más lejano.
Excepcion	No se encuentran talleres dentro del radio indicado: el sistema devuelve una lista vacía e informa que no hay talleres disponibles en esa área.

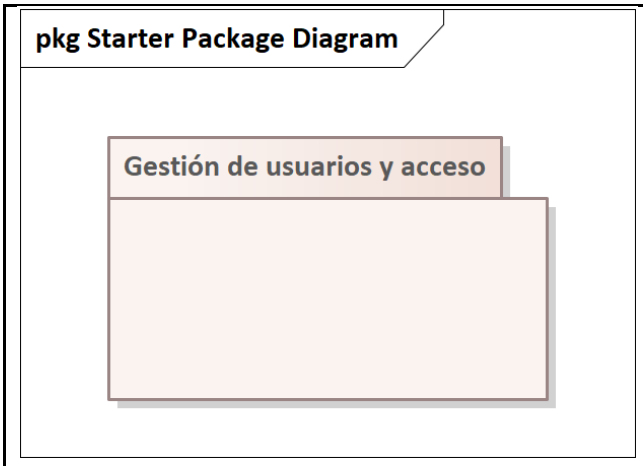
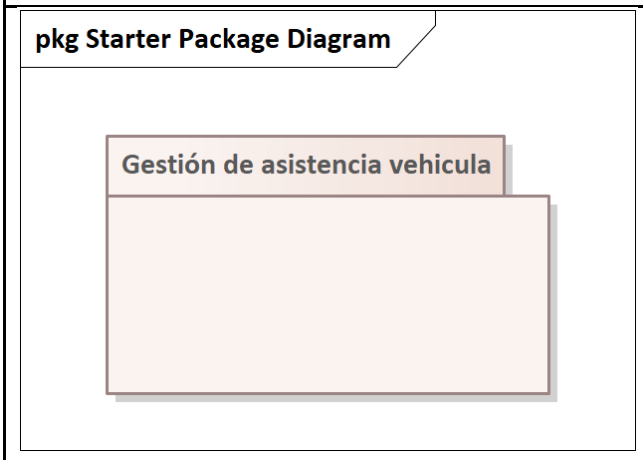
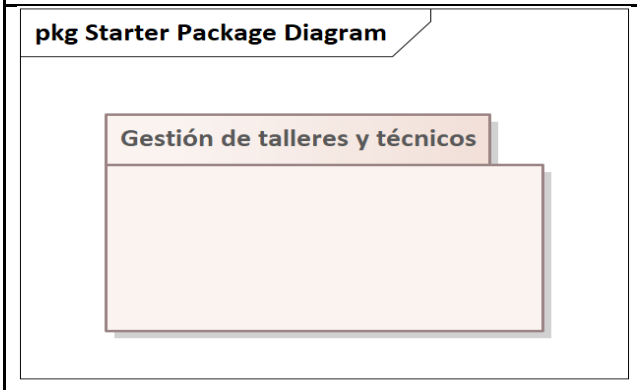
2.1.4 Estructurar el modelo de casos de uso

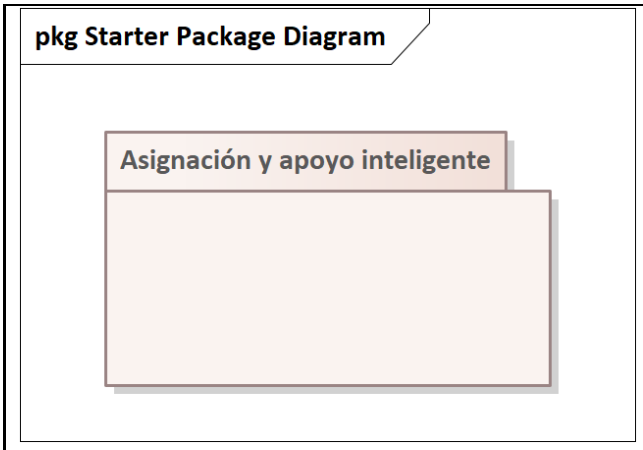
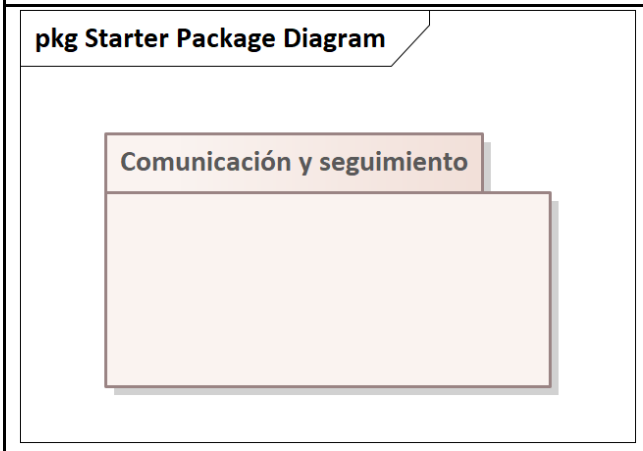
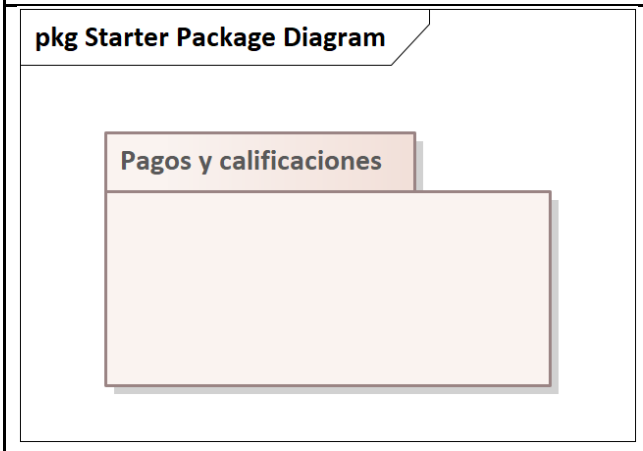


2.2 FT:ANALISIS

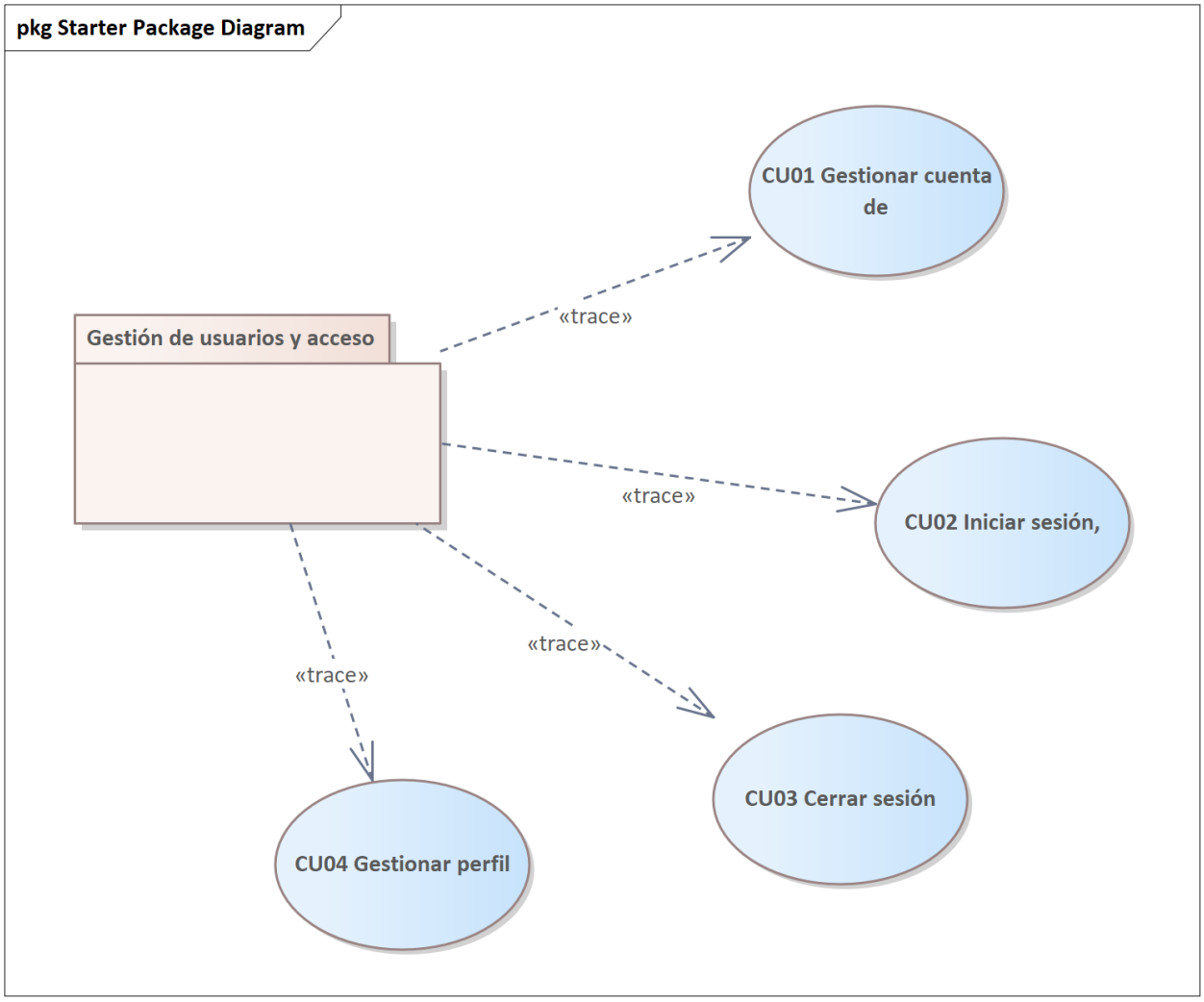
2.2.1 Analisis de arquitectura

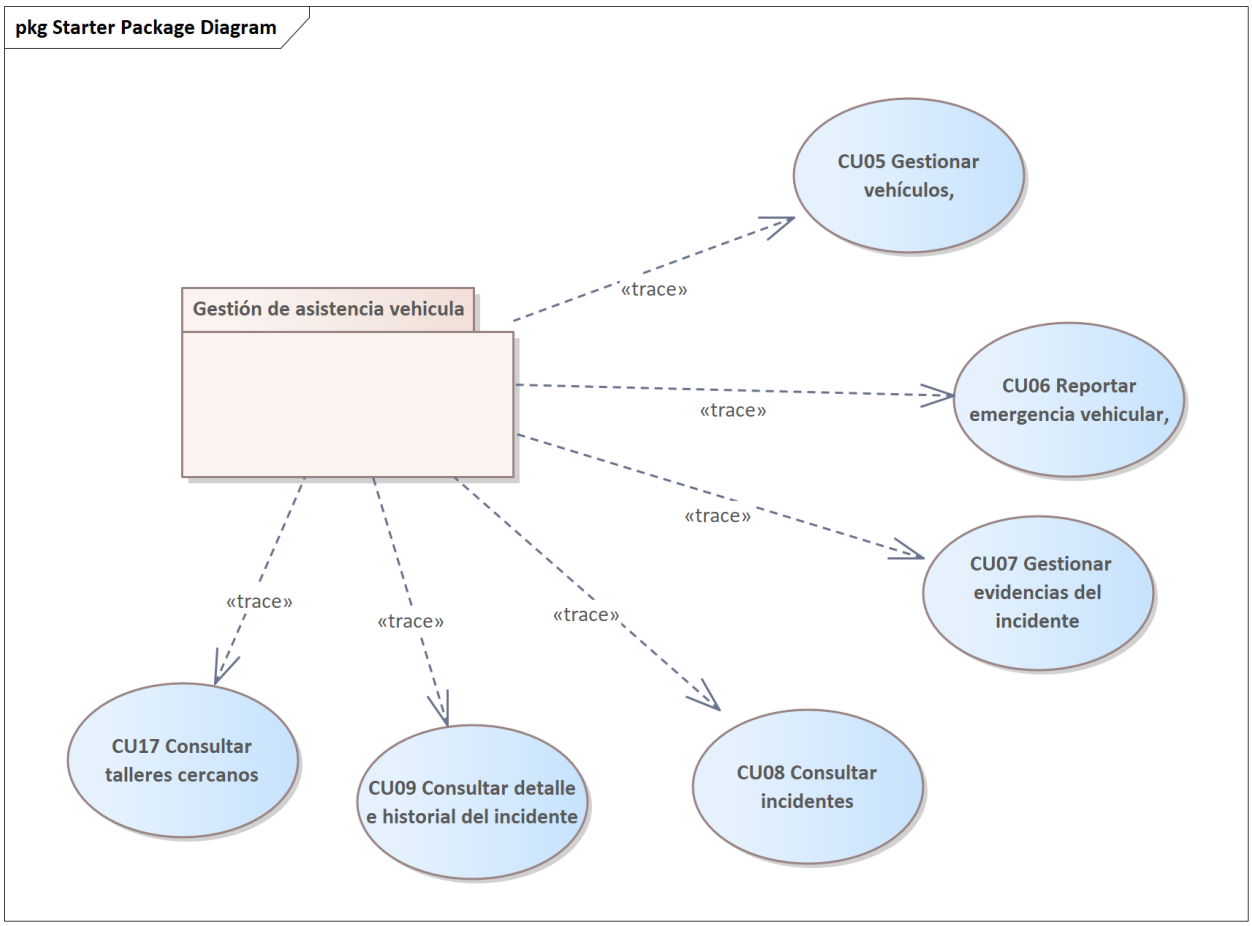
2.2.1.1 Identificar Paquetes

	Este paquete se encarga de la administración de cuentas dentro del sistema. Incluye el registro, inicio y cierre de sesión, gestión de perfiles y control de roles de acceso para administrador, cliente, taller y técnico.
	Este paquete se encarga del proceso principal de atención de emergencias. Incluye la gestión de vehículos, el registro de incidentes, el manejo de evidencias, así como la consulta del detalle e historial de cada caso.
	.Este paquete se encarga de administrar los talleres registrados y sus técnicos asociados. Incluye la gestión de disponibilidad, datos operativos, asignación de personal y seguimiento de los trabajos atendidos.

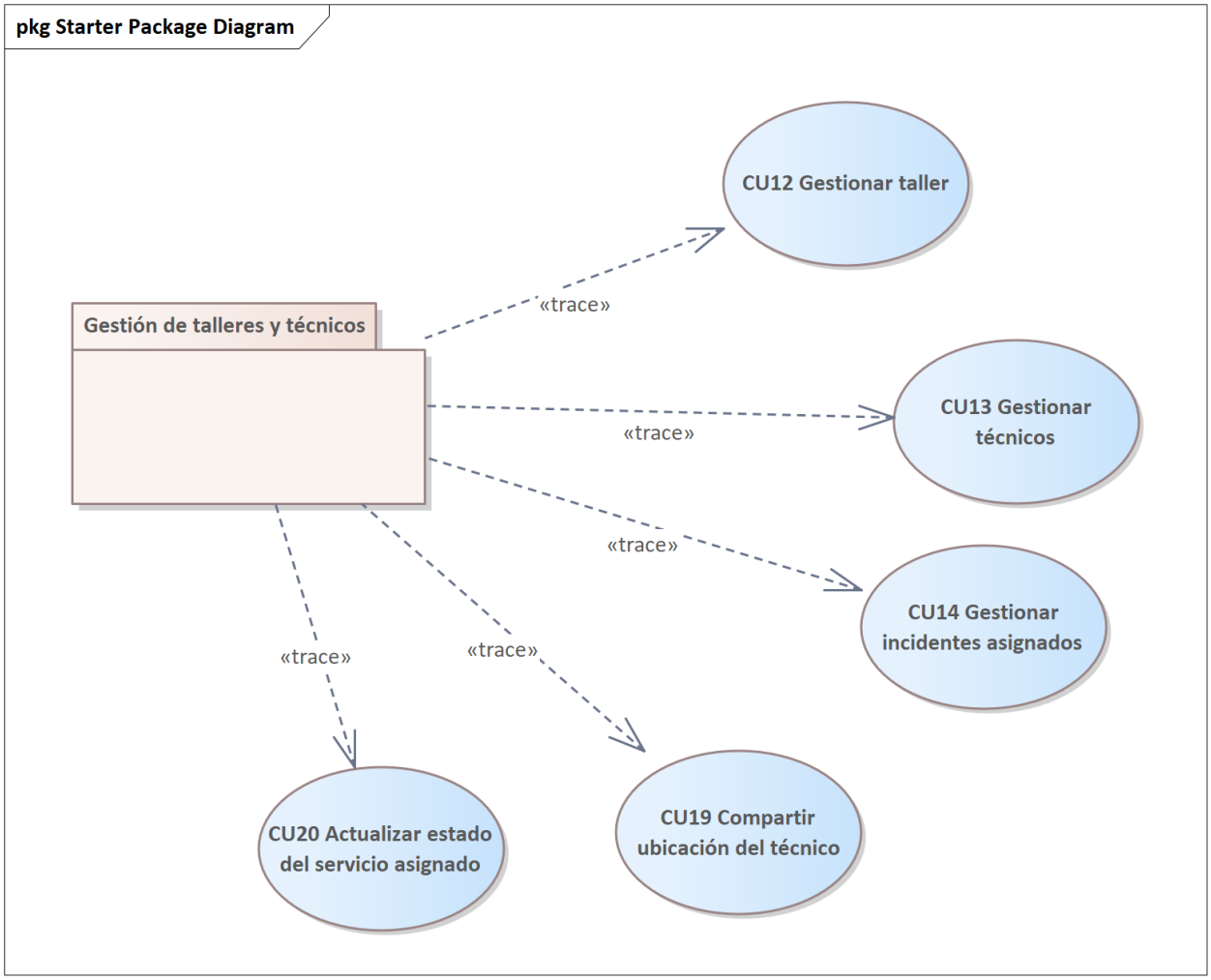
	Este paquete se encarga de apoyar la toma de decisiones en la atención de incidentes. Incluye la selección de talleres candidatos, la asignación automática y el procesamiento del incidente mediante inteligencia artificial.
	Este paquete se encarga de mantener informados a los actores del sistema durante la atención de una emergencia. Incluye notificaciones, mensajería y comunicación en tiempo real para el seguimiento del servicio.
	Este paquete se encarga de administrar los pagos realizados por los servicios y la evaluación posterior de la atención. Incluye el registro de pagos, comisiones, reseñas y calificaciones del servicio recibido.

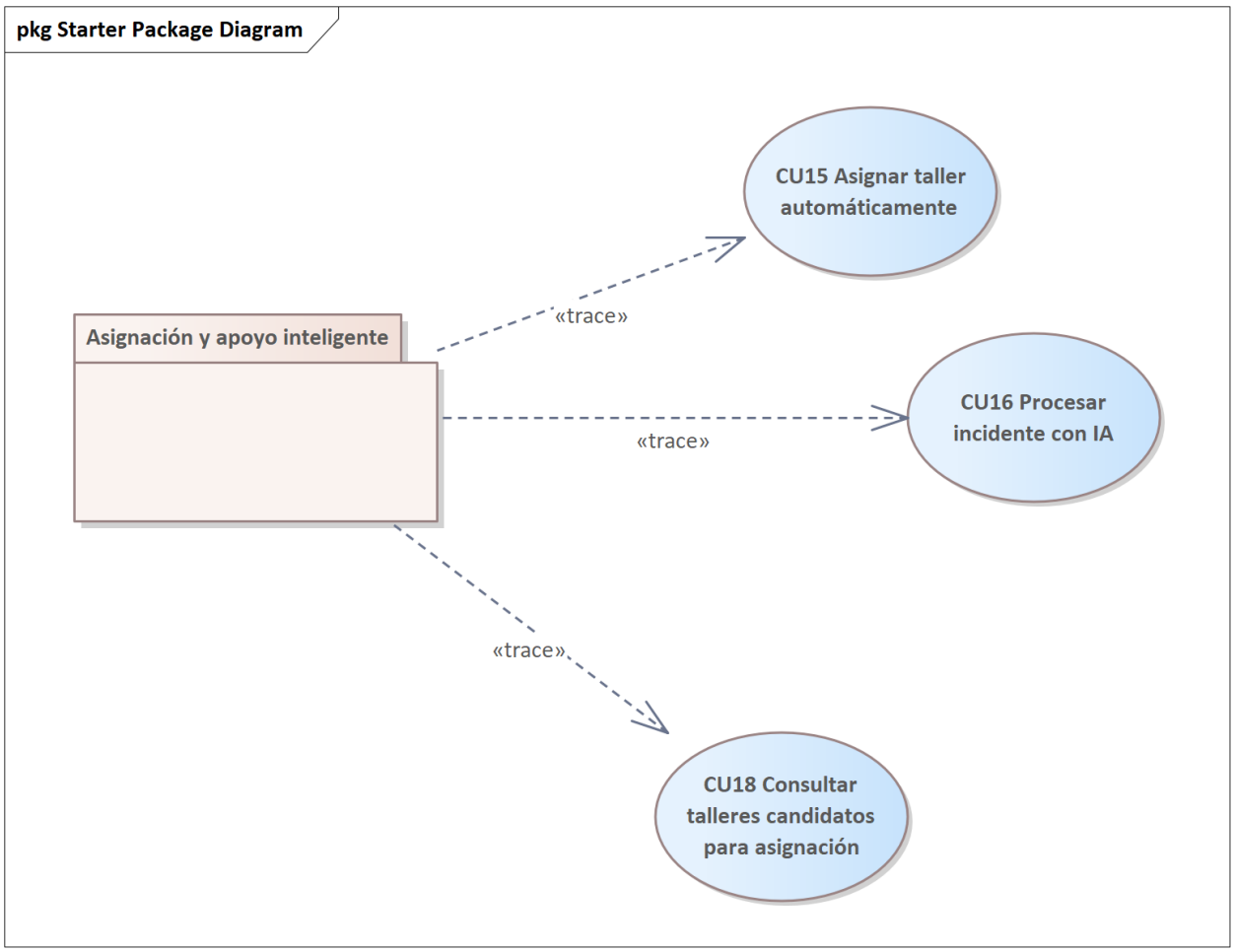
2.2.1.2 Relacionar Paquetes y Casos de Uso

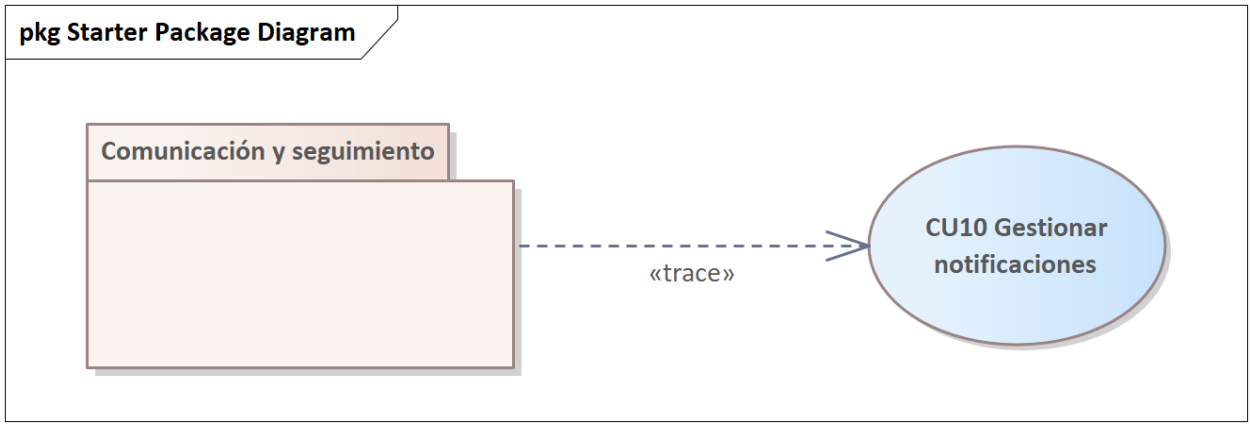


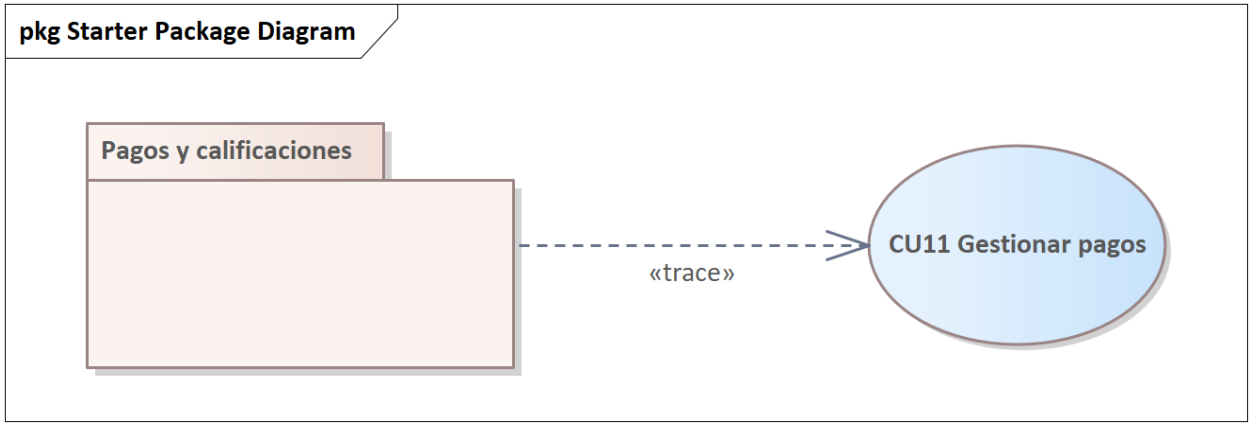

pkg Starter Package Diagram
CU05 Gestionar vehículos,
«trace»
Gestión de asistencia vehicula
CU06 Reportar emergencia vehicular,
«trace»
«trace»
CU07 Gestionar evidencias del incidente
«trace»
«trace»
«trace»
CU17 Consultar talleres cercanos
CU09 Consultar detalle e historial del incidente
CU08 Consultar incidentes

\


pkg Starter Package Diagram
CU12 Gestionar taller
«trace»
Gestión de talleres y técnicos
CU13 Gestionar técnicos
«trace»
«trace»
CU14 Gestionar incidentes asignados
«trace»
«trace»
CU20 Actualizar estado del servicio asignado
CU19 Compartir ubicación del técnico

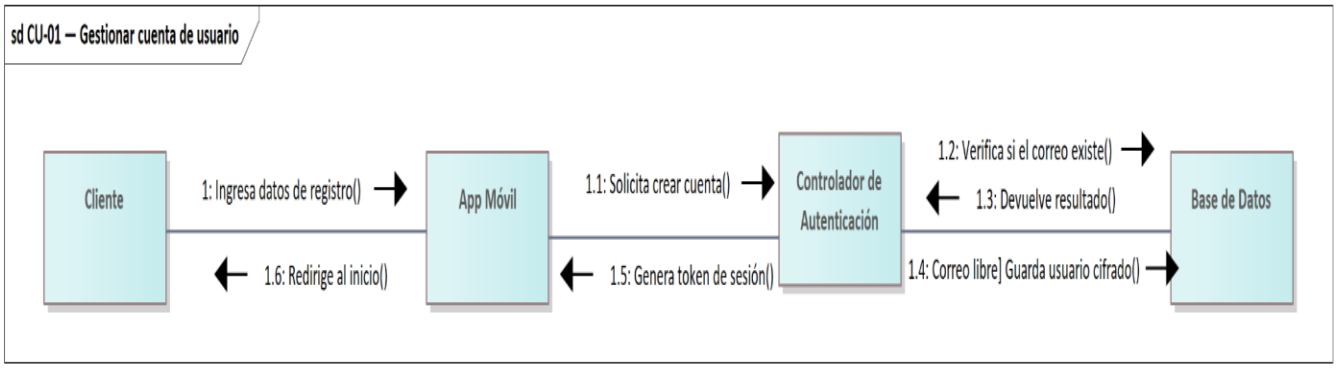

pkg Starter Package Diagram
CU15 Asignar taller automáticamente
Asignación y apoyo inteligente
«trace»
CU16 Procesar incidente con IA
«trace»
«trace»
CU18 Consultar talleres candidatos para asignación

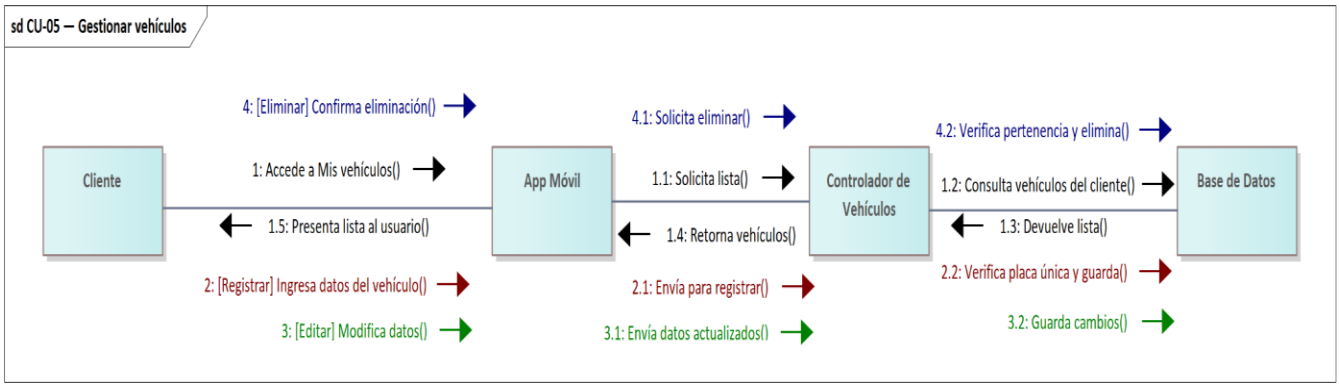

pkg Starter Package Diagram
Comunicación y seguimiento
«trace»
CU10 Gestionar notificaciones

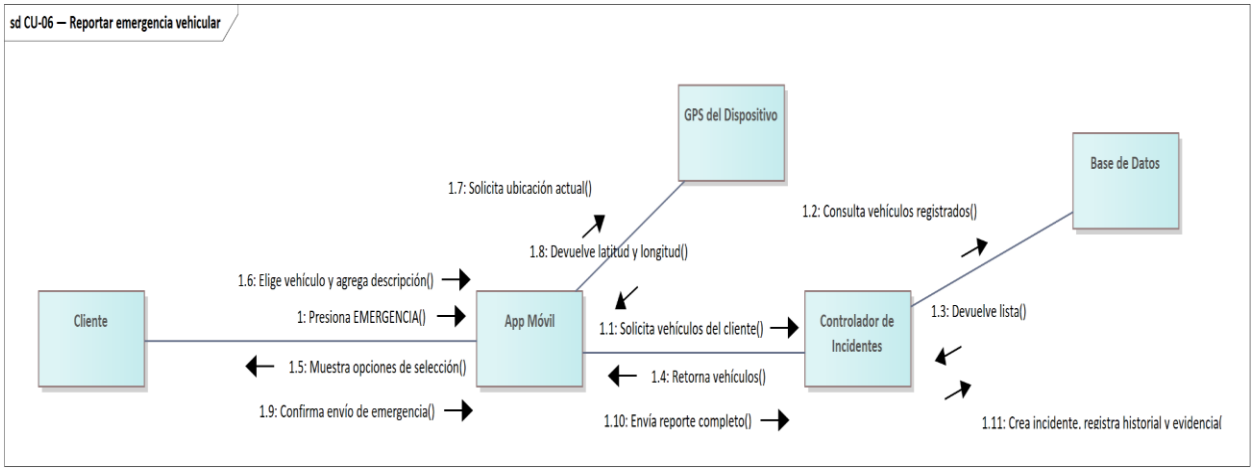


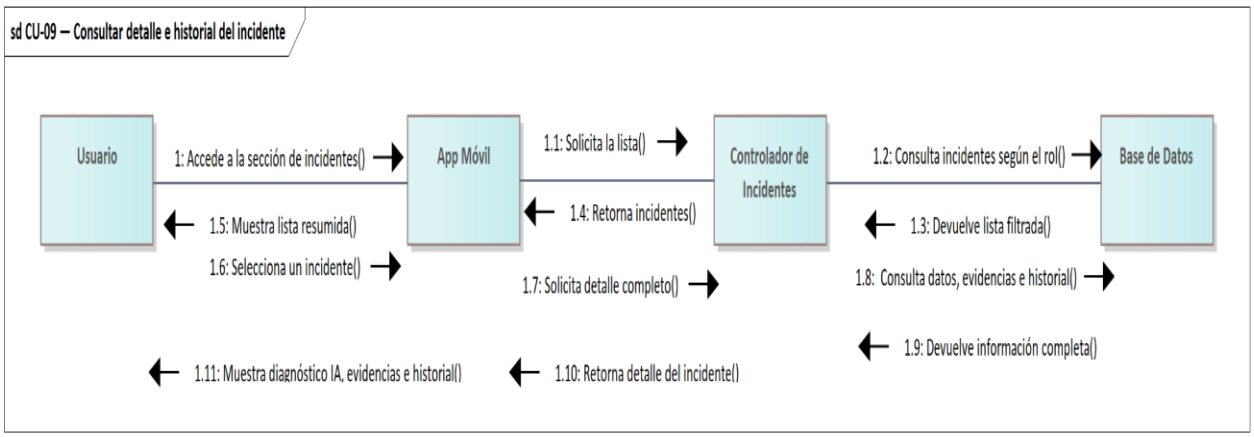
2.2.2 Analisis de Casos de Uso

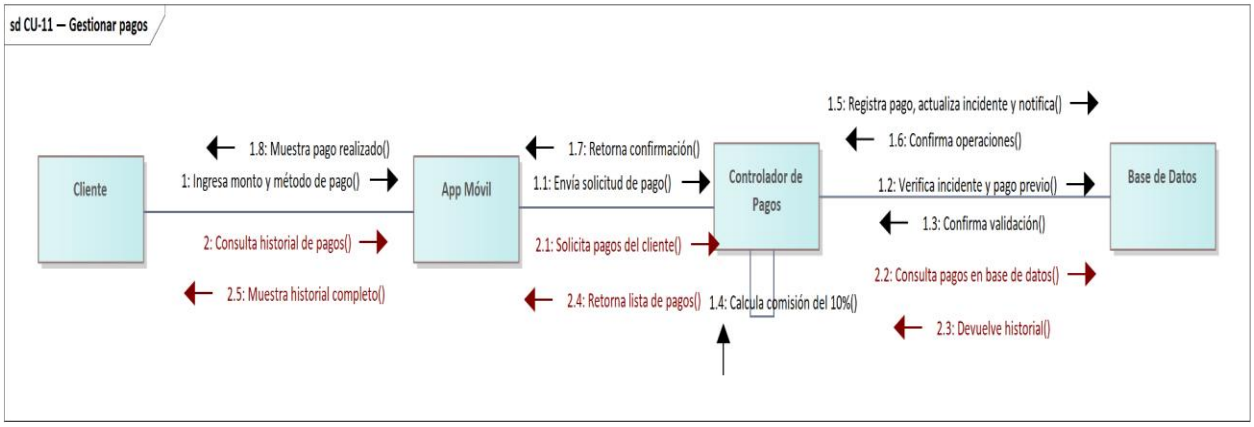
2.2.2.1 Diagramas de Colaboracion/Comunicacion

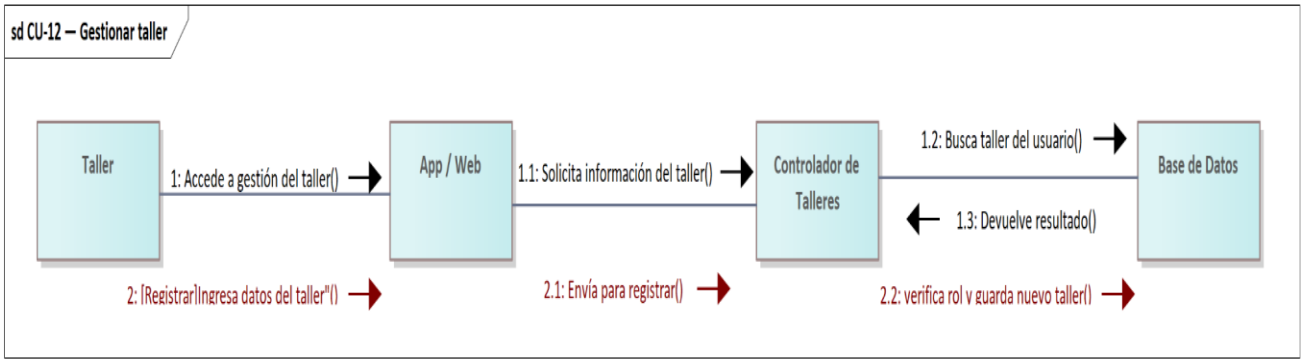


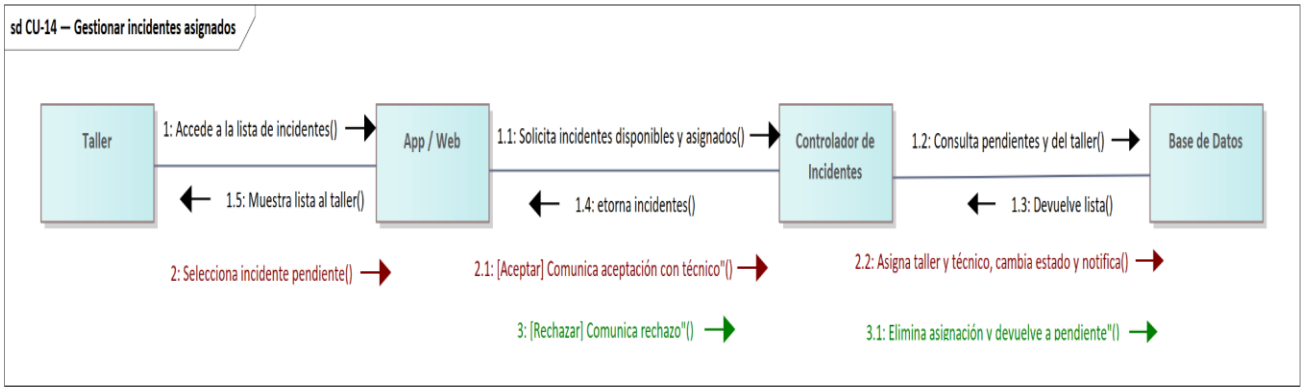



sd CU-06 – Reportar emergencia vehicular
GPS del Dispositivo
Base de Datos
1.7: Solicita ubicación actual()
1.2: Consulta vehículos registrados()
1.8: Devuelve latitud y longitud()
1.6: Elige vehículo y agrega descripción()
Cliente
1: Presiona EMERGENCIA()
App Móvil
1.1: Solicita vehículos del cliente()
Controlador de Incidentes
1.3: Devuelve lista()
1.5: Muestra opciones de selección()
1.4: Retorna vehículos()
1.9: Confirma envío de emergencia()
1.10: Envía reporte completo()
1.11: Crea incidente, registra historial y evidencia(

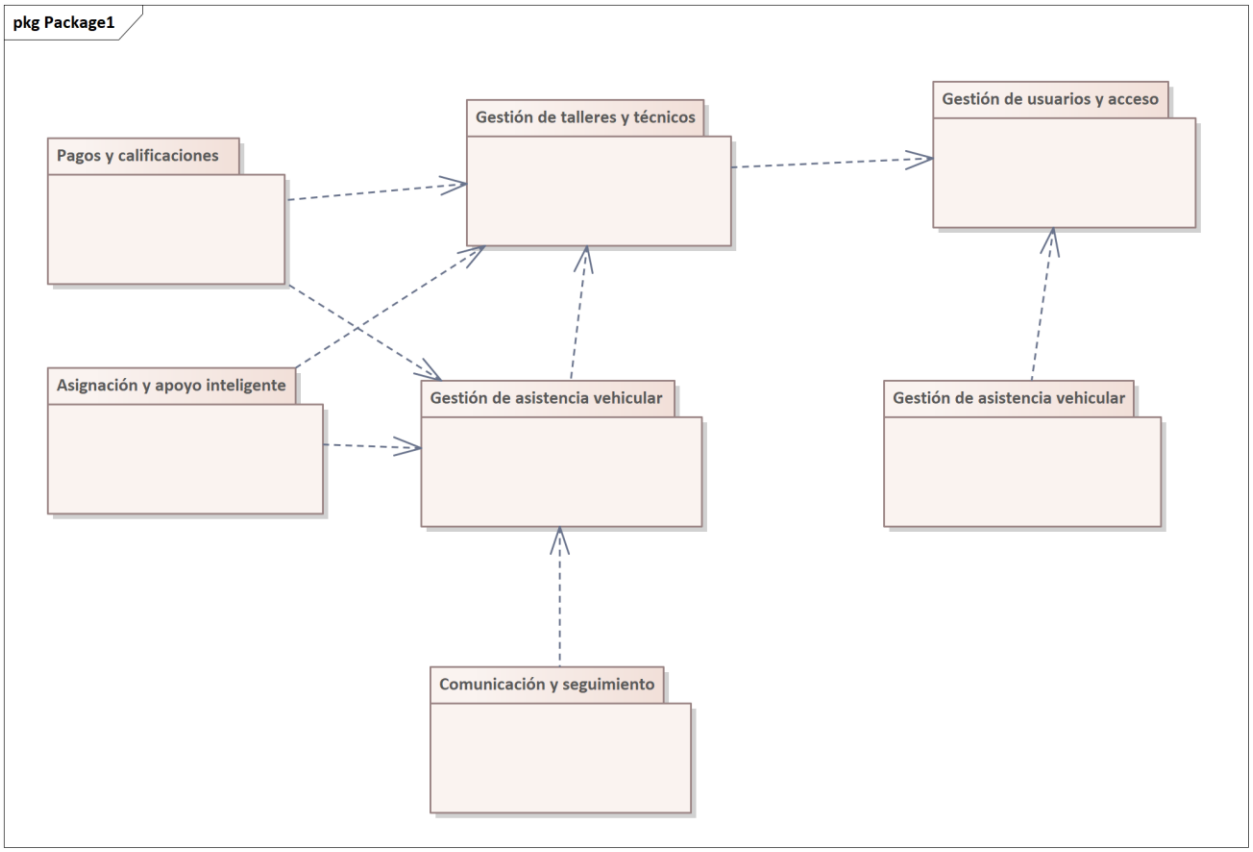

sd CU-09 – Consultar detalle e historial del incidente
Usuario
1: Accede a la sección de incidentes()
App Móvil
1.1: Solicita la lista()
Controlador de Incidentes
1.2: Consulta incidentes según el rol()
Base de Datos
1.5: Muestra lista resumida()
1.4: Retorna incidentes()
1.3: Devuelve lista filtrada()
1.6: Selecciona un incidente()
1.7: Solicita detalle completo()
1.8: Consulta datos, evidencias e historial()
1.9: Devuelve información completa()
1.11: Muestra diagnóstico IA, evidencias e historial()
1.10: Retorna detalle del incidente()


sd CU-11 – Gestionar pagos
1.5: Registra pago, actualiza incidente y notifica()
1.8: Muestra pago realizado()
1.7: Retorna confirmación()
1.6: Confirma operaciones()
Cliente
1: Ingresa monto y método de pago()
App Móvil
1.1: Envía solicitud de pago()
Controlador de Pagos
1.2: Verifica incidente y pago previo()
Base de Datos
1.3: Confirma validación()
2: Consulta historial de pagos()
2.1: Solicita pagos del cliente()
2.2: Consulta pagos en base de datos()
2.5: Muestra historial completo()
2.4: Retorna lista de pagos()
1.4: Calcula comisión del 10%()
2.3: Devuelve historial()


sd CU-12 – Gestionar taller
Taller
1: Accede a gestión del taller()
App / Web
1.1: Solicita información del taller()
Controlador de Talleres
1.2: Busca taller del usuario()
1.3: Devuelve resultado()
Base de Datos
2: [Registrar]Ingresa datos del taller"()
2.1: Envía para registrar()
2.2: verifica rol y guarda nuevo taller()


sd CU-14 – Gestionar incidentes asignados
Taller
1: Accede a la lista de incidentes()
App / Web
1.1: Solicita incidentes disponibles y asignados()
Controlador de Incidentes
1.2: Consulta pendientes y del taller()
1.3: Devuelve lista()
Base de Datos
1.4: etorna incidentes()
1.5: Muestra lista al taller()
2: Selecciona incidente pendiente()
2.1: [Aceptar] Comunica aceptación con técnico"()
2.2: Asigna taller y técnico, cambia estado y notifica()
3: [Rechazar] Comunica rechazo"()
3.1: Elimina asignación y devuelve a pendiente"()

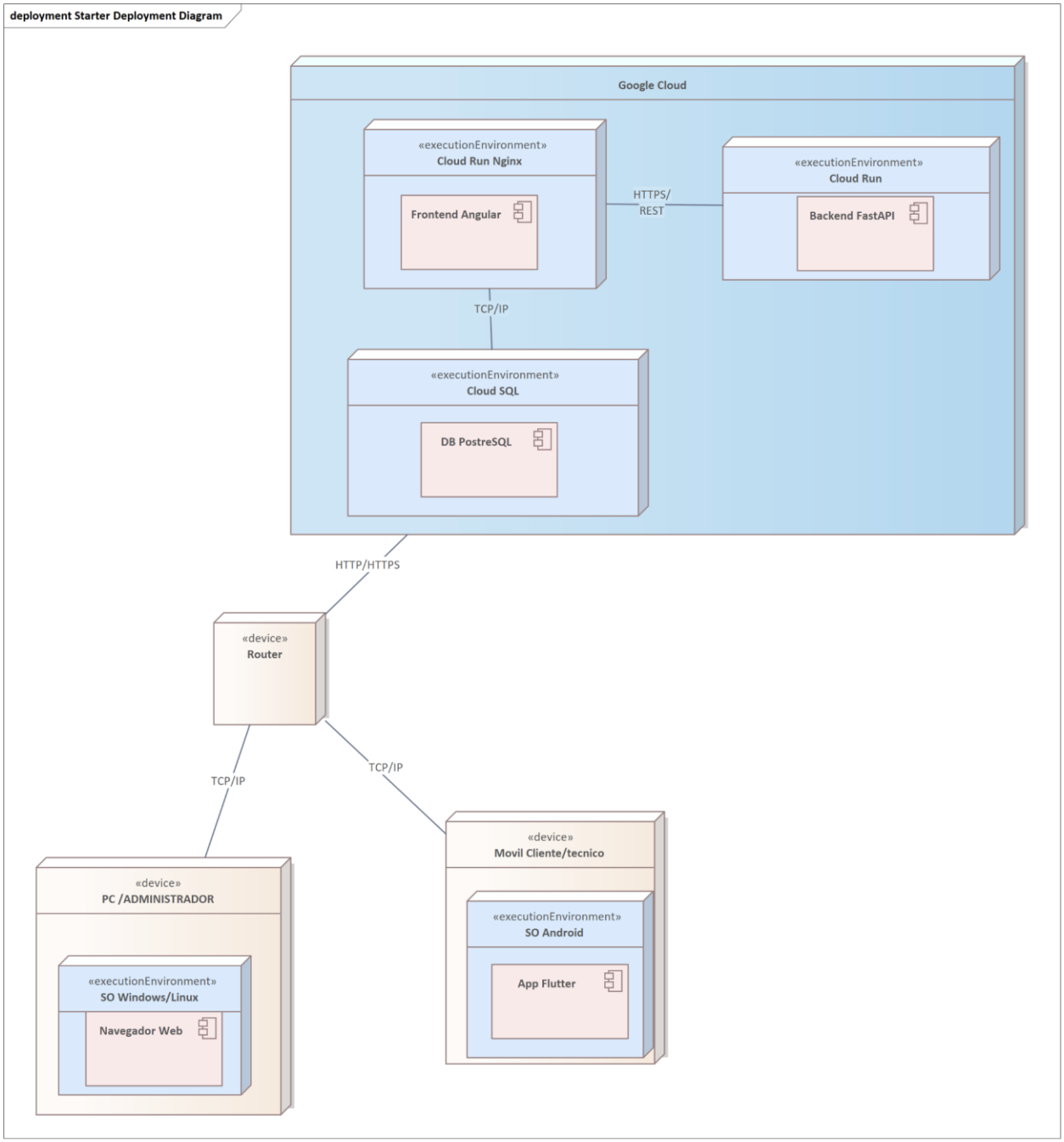
2.2.3 Analisis de Paquetes



2.3 FT: Diseno

2.3.1 Diseno de Arquitectura

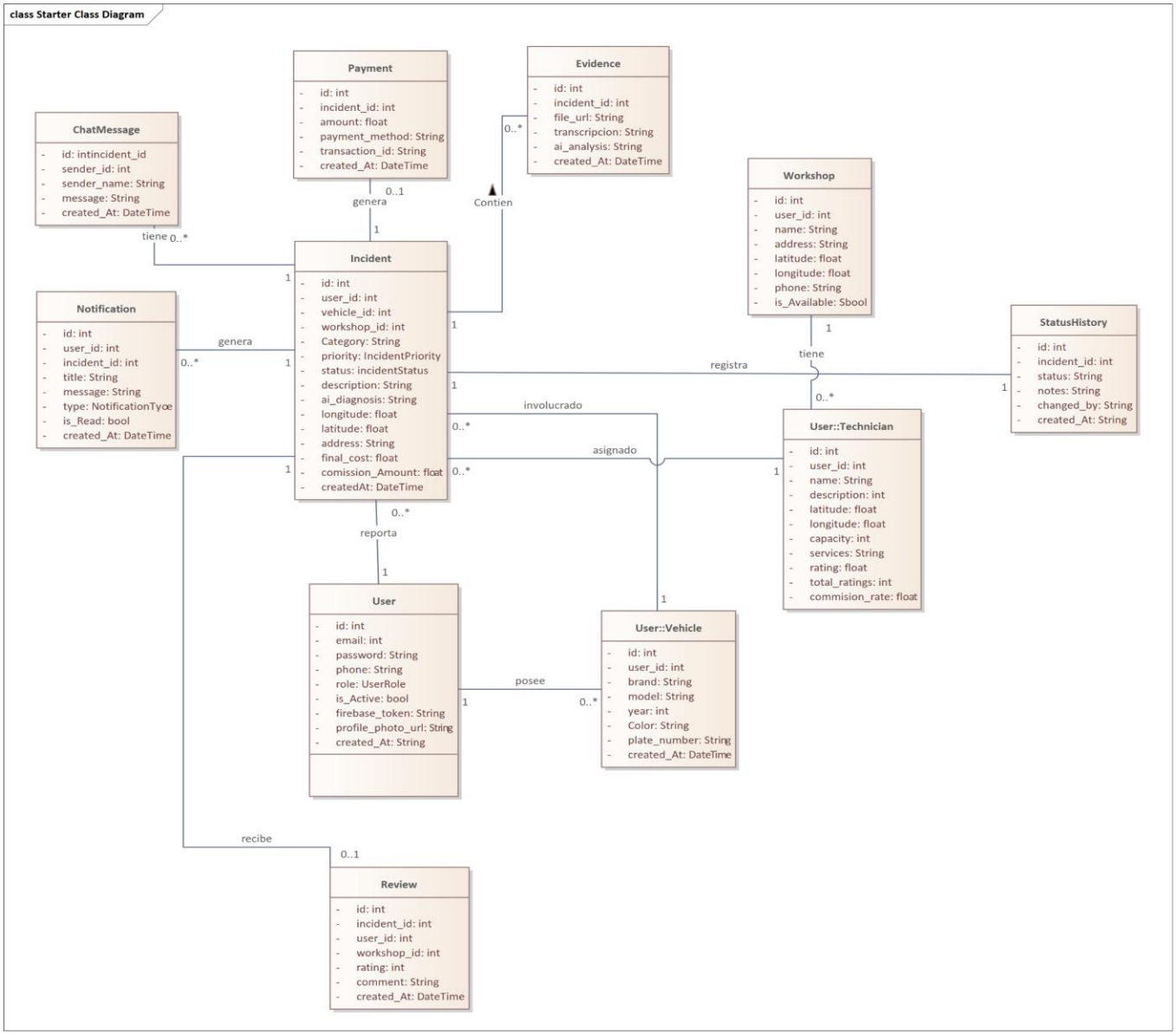
2.3.1.1 Diseno Fisico



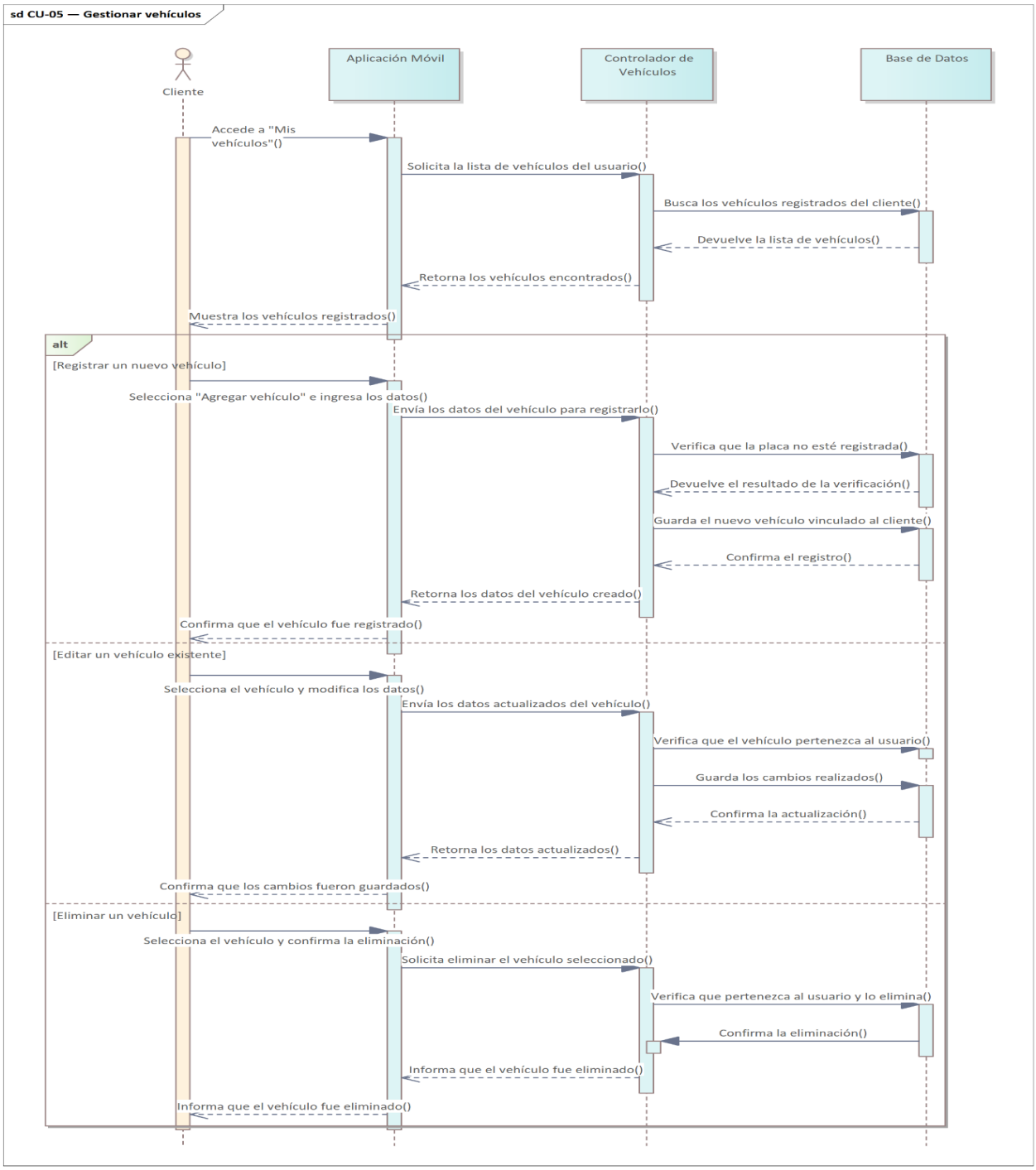
2.3.2 Diseno de Datos

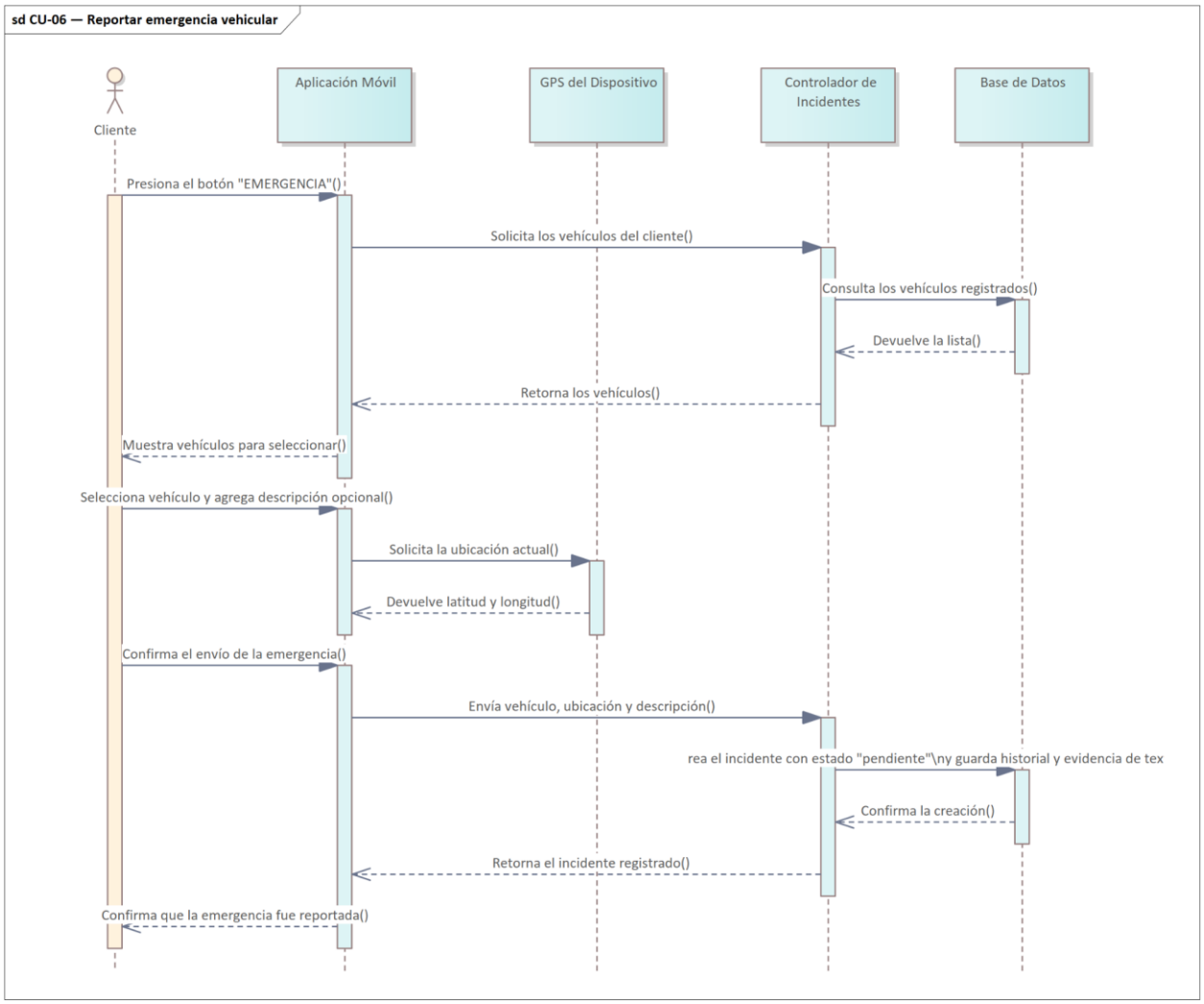
2.3.2.1 Diseno datos Logicos

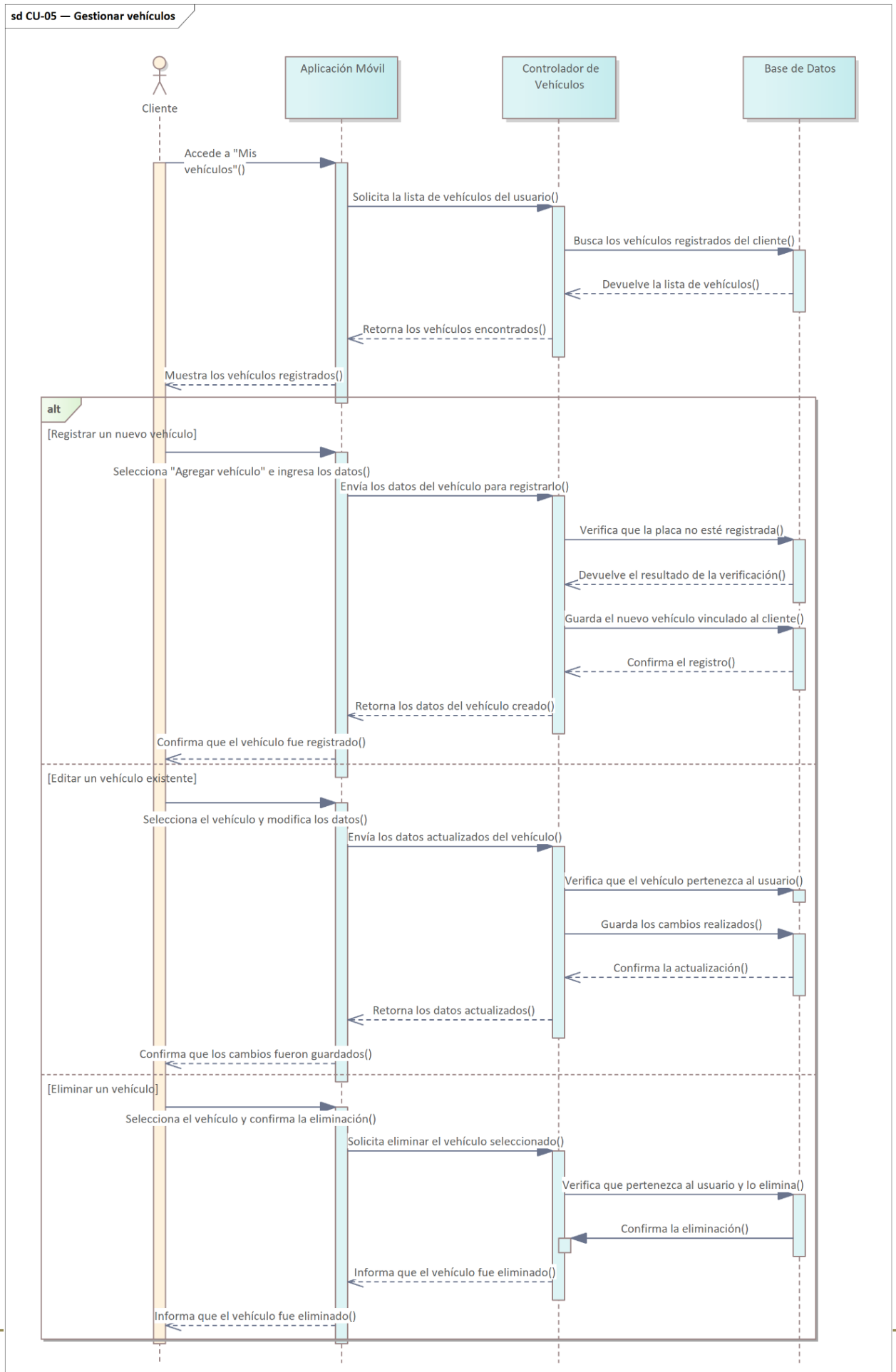
2.3.2.1.1 Diagrama de Clases

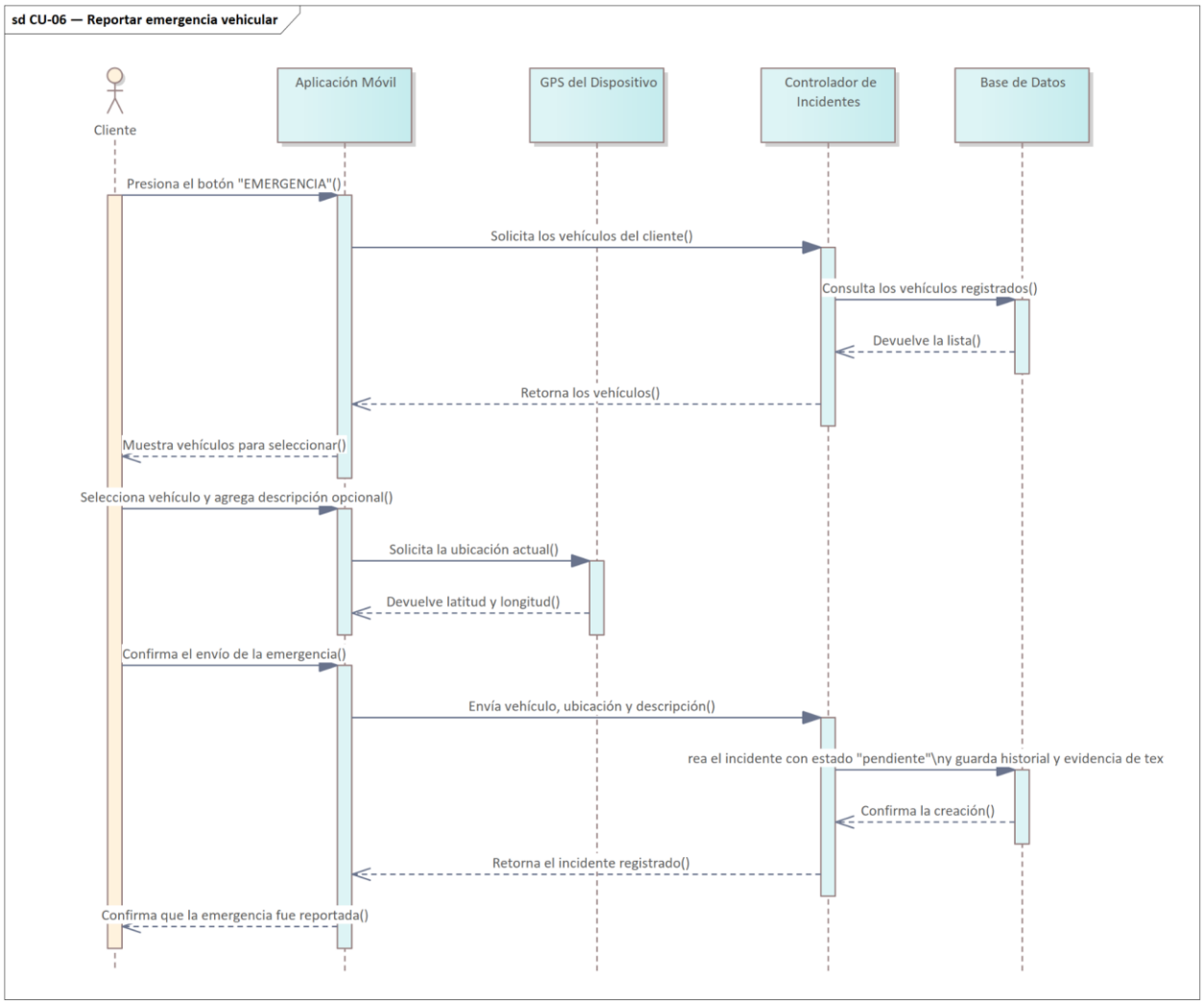


2.3.3 Diagrama de secuencia


sd CU-05 — Gestionar vehículos
Cliente
Aplicación Móvil
Controlador de Vehículos
Base de Datos
Accede a "Mis vehículos"()
Solicita la lista de vehículos del usuario()
Busca los vehículos registrados del cliente()
Devuelve la lista de vehículos()
Retorna los vehículos encontrados()
Muestra los vehículos registrados()
alt
[Registrar un nuevo vehículo]
Selecciona "Agregar vehículo" e ingresa los datos()
Envía los datos del vehículo para registrarlo()
Verifica que la placa no esté registrada()
Devuelve el resultado de la verificación()
Guarda el nuevo vehículo vinculado al cliente()
Confirma el registro()
Retorna los datos del vehículo creado()
Confirma que el vehículo fue registrado()
[Editar un vehículo existente]
Selecciona el vehículo y modifica los datos()
Envía los datos actualizados del vehículo()
Verifica que el vehículo pertenezca al usuario()
Guarda los cambios realizados()
Confirma la actualización()
Retorna los datos actualizados()
Confirma que los cambios fueron guardados()
[Eliminar un vehículo]
Selecciona el vehículo y confirma la eliminación()
Solicita eliminar el vehículo seleccionado()
Verifica que pertenezca al usuario y lo elimina()
Confirma la eliminación()
Informa que el vehículo fue eliminado()
Informa que el vehículo fue eliminado()


sd CU-06 — Reportar emergencia vehicular
Cliente
Aplicación Móvil
GPS del Dispositivo
Controlador de Incidentes
Base de Datos
Presiona el botón "EMERGENCIA"()
Solicita los vehículos del cliente()
Consulta los vehículos registrados()
Devuelve la lista()
Retorna los vehículos()
Muestra vehículos para seleccionar()
Selecciona vehículo y agrega descripción opcional()
Solicita la ubicación actual()
Devuelve latitud y longitud()
Confirma el envío de la emergencia()
Envía vehículo, ubicación y descripción()
rea el incidente con estado "pendiente"\ny guarda historial y evidencia de tex
Confirma la creación()
Retorna el incidente registrado()
Confirma que la emergencia fue reportada()


sd CU-05 — Gestionar vehículos
Cliente
Aplicación Móvil
Controlador de Vehículos
Base de Datos
Accede a "Mis vehículos"()
Solicita la lista de vehículos del usuario()
Busca los vehículos registrados del cliente()
Devuelve la lista de vehículos()
Retorna los vehículos encontrados()
Muestra los vehículos registrados()
alt
[Registrar un nuevo vehículo]
Selecciona "Agregar vehículo" e ingresa los datos()
Envía los datos del vehículo para registrarlo()
Verifica que la placa no esté registrada()
Devuelve el resultado de la verificación()
Guarda el nuevo vehículo vinculado al cliente()
Confirma el registro()
Retorna los datos del vehículo creado()
Confirma que el vehículo fue registrado()
[Editar un vehículo existente]
Selecciona el vehículo y modifica los datos()
Envía los datos actualizados del vehículo()
Verifica que el vehículo pertenezca al usuario()
Guarda los cambios realizados()
Confirma la actualización()
Retorna los datos actualizados()
Confirma que los cambios fueron guardados()
[Eliminar un vehículo]
Selecciona el vehículo y confirma la eliminación()
Solicita eliminar el vehículo seleccionado()
Verifica que pertenezca al usuario y lo elimina()
Confirma la eliminación()
Informa que el vehículo fue eliminado()
Informa que el vehículo fue eliminado()



2.4 FT:Implementacion

2.4.1 Elección de plataforma de Desarrollo de Software

La plataforma seleccionada para el presente proyecto combina Python para el backend, Angular para la aplicación web administrativa y Flutter para la aplicación móvil. Esta elección responde a la necesidad de construir una solución integral para la gestión de auxilio mecánico vehicular, con capacidades de atención en tiempo real, administración operativa, geolocalización, notificaciones y una experiencia diferenciada según el rol del usuario.

En el backend se emplea FastAPI, un framework moderno de alto rendimiento para Python, adecuado para el desarrollo de APIs REST robustas, tipadas y bien

documentadas. Para la web se utiliza Angular, orientado a la construcción de interfaces estructuradas y mantenibles para la administración de talleres y la supervisión de la plataforma. Para la capa móvil se trabaja con Flutter, lo que permite ofrecer una experiencia consistente y multiplataforma para clientes y técnicos/mecánicos desde una sola base de código.

Esta combinación tecnológica se alinea con los requisitos funcionales del sistema: registro y autenticación de usuarios, gestión de incidentes, carga de evidencias como fotos y audio, integración con servicios de inteligencia artificial, notificaciones en tiempo real, seguimiento de estados del servicio y operación diferenciada entre cliente, técnico, taller y administrador.

2.4.1.1 Lenguajes y Frameworks de Programación

Para la implementación del sistema se seleccionaron distintos lenguajes y frameworks según la naturaleza de cada componente:

Backend: Python con FastAPI

Python fue elegido para el backend debido a su productividad, legibilidad y amplio ecosistema para desarrollo web, procesamiento de datos e integración con servicios de inteligencia artificial. En particular, FastAPI ofrece ventajas importantes para este proyecto:

Alto rendimiento

FastAPI está basado en ASGI y ofrece un desempeño competitivo para APIs modernas.

Permite manejar de forma eficiente múltiples solicitudes concurrentes, lo cual es importante en una plataforma con incidentes, notificaciones y actualizaciones en tiempo real.

Tipado y validación

Usa anotaciones de tipos de Python y modelos basados en Pydantic, lo que permite validar entradas y salidas de forma clara.

Esto mejora la confiabilidad de los endpoints y reduce errores en la comunicación entre cliente y servidor.

Productividad y mantenibilidad

La sintaxis de Python favorece un desarrollo rápido y comprensible.

La organización del proyecto en routers, schemas, models y services facilita el mantenimiento y la evolución del sistema.

Integración con IA y servicios externos

Python cuenta con excelente soporte para bibliotecas y SDKs vinculados a procesamiento de imágenes, audio, geolocalización y servicios de inteligencia artificial.

Esto resulta especialmente valioso en este proyecto, donde el sistema debe analizar evidencias del incidente y generar apoyo inteligente para la clasificación del problema.

Documentación automática

FastAPI genera documentación interactiva mediante OpenAPI/Swagger.

Esto simplifica las pruebas, el consumo de la API y la validación del sistema durante el desarrollo.

Frontend Web: TypeScript con Angular

Para la aplicación web se seleccionó Angular, utilizando TypeScript como lenguaje base. Esta elección se justifica porque la web está orientada a procesos administrativos y operativos de talleres y administradores de plataforma, donde se requiere estructura, escalabilidad y consistencia arquitectónica.

Arquitectura sólida

Angular proporciona una estructura clara basada en componentes, servicios, rutas y guards.

Esto facilita la implementación de módulos administrativos, paneles de gestión y controles por rol.

Uso de TypeScript

TypeScript aporta tipado estático, interfaces y mayor seguridad durante el desarrollo.

Esto mejora la mantenibilidad del frontend y reduce errores en el consumo de la API.

Escalabilidad

La naturaleza modular de Angular favorece el crecimiento del sistema.

Es especialmente útil en una aplicación con múltiples vistas como incidentes, historial, técnicos, reportes, usuarios y talleres.

Experiencia administrativa

Angular resulta apropiado para construir interfaces de trabajo más que páginas promocionales.

Esto coincide con la necesidad del proyecto de ofrecer herramientas operativas claras para talleres y administradores.

Aplicación Móvil: Dart con Flutter

Para la aplicación móvil se utiliza Flutter con lenguaje Dart. La selección de esta tecnología responde a la necesidad de brindar una experiencia móvil única para clientes/conductores y técnicos/mecánicos, manteniendo un desarrollo multiplataforma eficiente.

Multiplataforma

Flutter permite desplegar la misma aplicación en Android, iOS, web y escritorio a partir de una sola base de código.Esto reduce tiempo de desarrollo y esfuerzo de mantenimiento.

Interfaz rica y consistente

Flutter ofrece un sistema de widgets flexible y de alto rendimiento.

Esto permite crear pantallas fluidas para reportar emergencias, ver trabajos asignados, consultar estados y gestionar notificaciones.

Integración con funcionalidades del dispositivo

El proyecto requiere acceso a geolocalización, selección de imágenes, reproducción y grabación de audio, almacenamiento seguro y notificaciones push.

Flutter dispone de paquetes maduros para estas capacidades, facilitando la construcción de la experiencia móvil.

Adecuado para usuarios en campo

Los técnicos necesitan una interfaz rápida, clara y optimizada para movilidad.

Flutter permite construir flujos ágiles para seguimiento de rutas, cambio de estados del servicio y consulta de trabajos asignados.

2.4.1.2 DBMS (Database Management System)

Para el presente proyecto se ha seleccionado PostgreSQL como sistema de gestión de bases de datos. Esta elección responde a la necesidad de contar con una plataforma robusta, confiable y escalable para gestionar la información central del sistema, incluyendo usuarios, vehículos, talleres, técnicos, incidentes, evidencias, pagos, reseñas y notificaciones.

PostgreSQL es una solución relacional de código abierto ampliamente utilizada en sistemas empresariales y aplicaciones de misión crítica. Su adopción en este proyecto se fundamenta en las siguientes razones:

Confiabilidad e integridad

PostgreSQL implementa transacciones ACID, lo cual garantiza consistencia y seguridad de los datos.

Esto es esencial en un sistema donde distintos actores interactúan sobre la misma información operativa.

Modelado relacional adecuado

La naturaleza del proyecto involucra múltiples entidades relacionadas entre sí: cliente, vehículo, incidente, taller, técnico, pago y reseña.

PostgreSQL permite representar estas relaciones de forma natural y eficiente.

Escalabilidad y rendimiento

Es una base de datos preparada para crecer en volumen de información y número de operaciones.

Esto resulta importante si la plataforma incorpora más talleres, usuarios e incidentes en el tiempo.

Compatibilidad con el backend

PostgreSQL se integra de manera adecuada con SQLAlchemy, utilizado en el backend desarrollado con FastAPI.

Esta combinación facilita la definición de modelos, la persistencia de datos y la evolución del esquema de la base de datos.

Soporte para datos diversos

Además del modelo relacional clásico, PostgreSQL ofrece tipos avanzados y soporte para estructuras más flexibles cuando sea necesario.

Esto puede resultar útil para almacenar metadatos, evidencias o configuraciones futuras del sistema.

En conjunto, PostgreSQL constituye una base sólida para soportar la operación de la plataforma, asegurando integridad, trazabilidad y capacidad de crecimiento.

2.4.1.3 Sistemas Operativos

El sistema operativo seleccionado como referencia principal para la implementación y despliegue del proyecto es Linux. Esta decisión se fundamenta en su estabilidad, seguridad, eficiencia en entornos de servidor y amplia adopción en plataformas de nube como AWS y Microsoft Azure.

Linux representa una opción adecuada para ejecutar el backend, administrar servicios web, desplegar contenedores y garantizar portabilidad entre entornos de desarrollo, pruebas y producción. Entre las razones principales de esta elección se encuentran las siguientes:

Estabilidad

Linux es ampliamente utilizado en servidores y ambientes productivos por su comportamiento confiable bajo carga.

Esto es importante para una plataforma que debe mantener disponibilidad operativa para incidentes y notificaciones.

Seguridad

Proporciona mecanismos sólidos de permisos, aislamiento y administración del sistema.

Esto contribuye a proteger la información sensible de usuarios, autenticación, pagos e incidentes.

Compatibilidad con contenedores

El proyecto contempla Docker como mecanismo de empaquetado y despliegue del backend.

Linux ofrece integración nativa y eficiente con tecnologías de contenedorización.

Cloud-native

Los principales proveedores de nube están fuertemente orientados a Linux para despliegues de aplicaciones modernas.

Esto facilita una futura publicación del sistema en infraestructura escalable.

Eficiencia operativa

Linux permite una administración flexible de procesos, red y recursos del sistema.

Esto favorece un despliegue reproducible y sostenible en ambientes académicos y productivos.

Adicionalmente, el uso de Docker complementa esta elección al permitir encapsular la aplicación y sus dependencias en contenedores ligeros. De esta forma se mejora la portabilidad del sistema, se reducen problemas de configuración entre entornos y se facilita la puesta en marcha de la solución.

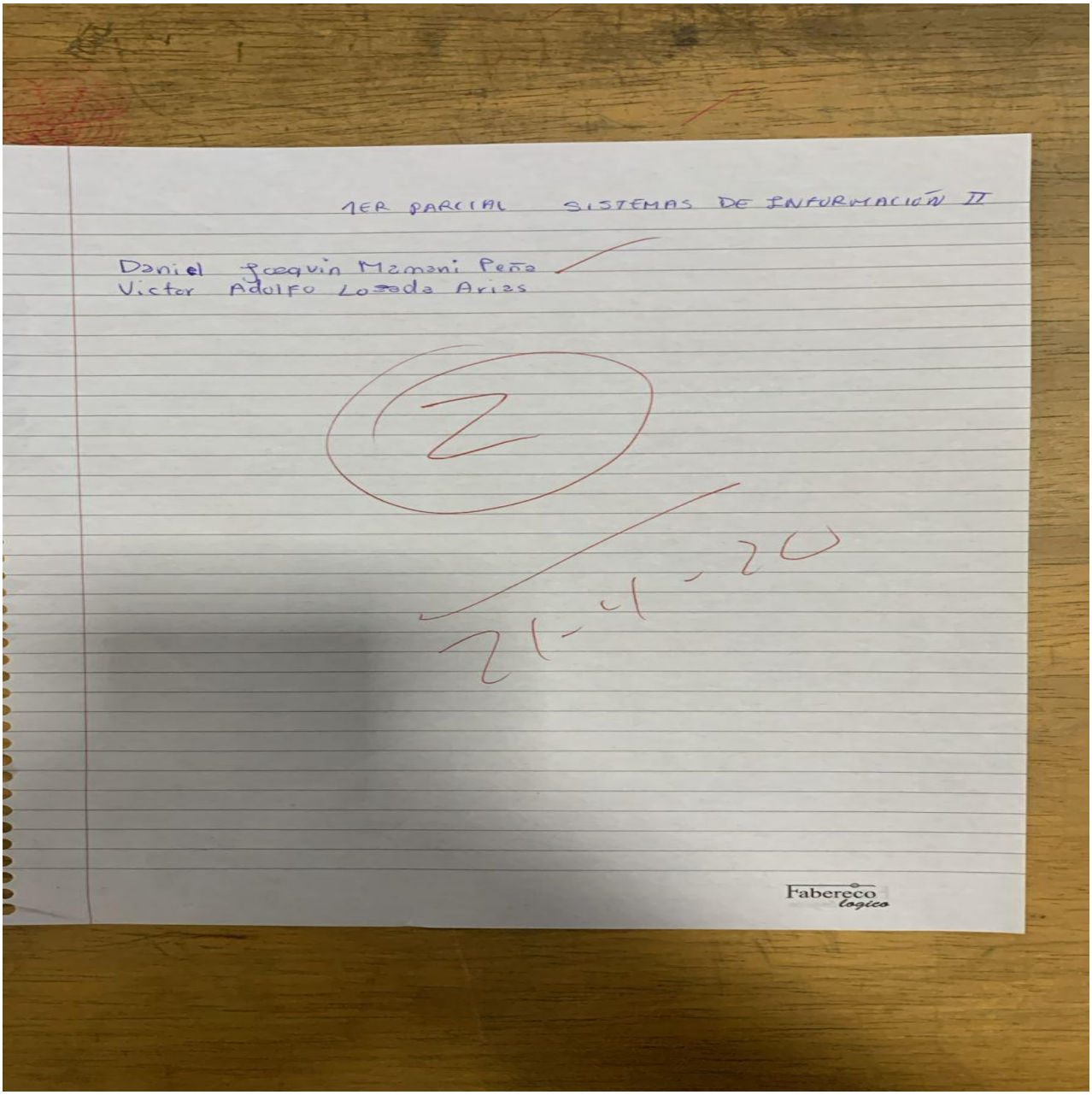
Si quieres, puedo convertir este texto a un formato más académico todavía, con redacción de tesis formal, o dejarlo listo con numeración exacta para pegarlo directamente en tu documento.

3 Bibliografia

- Rumbaugh, J., Jacobson, I., & Booch, G. (2005). The Unified Modeling Language Reference Manual (2nd ed.). Addison-Wesley Professional.
- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley Professional.
- Object Management Group. (2017). Unified Modeling Language (UML), Version 2.5.1. https://www.omg.org/spec/UML/2.5.1/
- Angular. (2024). Angular Documentation. https://angular.dev/
- FastAPI. (2024). FastAPI Documentation. https://fastapi.tiangolo.com/
- Flutter. (2024). Flutter Documentation. https://docs.flutter.dev/

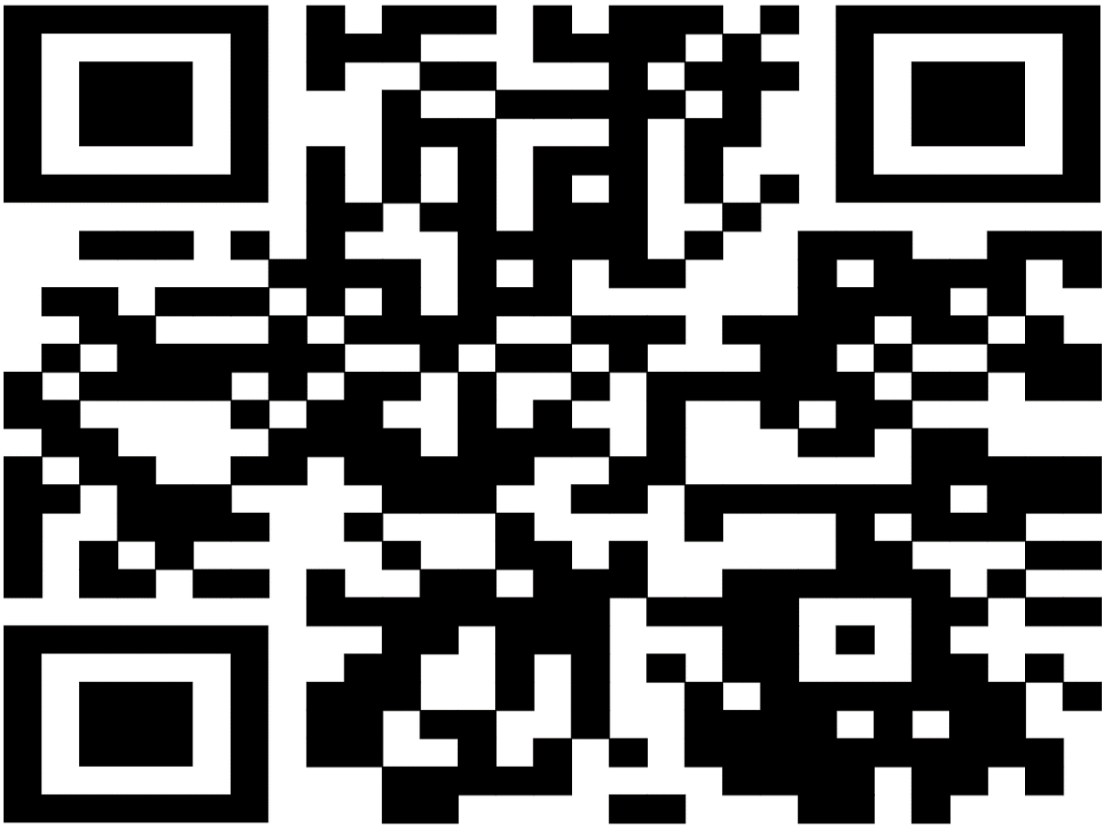
-

4 Anexo


1ER PARCIAL SISTEMAS DE INFORMACIÓN II
Daniel Joaquin Mamani Peña
Victor Adolfo Loayza Arias
2
21.1.20
Faberecó
logico

USUARIO	EMAIL	PASSWORD
ADMIN	admin@asistecar.com	12345678*
WORKSHOP	admin@servillantaexpress.com	12345678*
TECHNICIAN	jorge.pinto@elpiston.com	12345678*
CLIENT	cliente013@asistecar.demo	12345678*

REPOSITORIOS ASISTCAR



REPOSITORIO ASISTCAR



Repositorio WEB



REPOSITORIO MOBIL